

KING AND PEASANTS

ISMAEL CARRASCO MKHAZNI

Trabajo fin de Grado

Supervisado por Dr. Pablo Trinidad Martín-Arroyo



Universidad de Sevilla

mayo 2026

Publicado en mayo 2026 por

Ismael Carrasco Mkhazni

Copyright © MMXXVI

<http://www.lsi.us.es/~trinidad>

ismcarmkh@alum.us.es

Pon aquí cuestiones acerca del copyright

Yo, D. Ismael Carrasco Mkhazni con NIF número 15456902M,

DECLARO

mi autoría del trabajo que se presenta en la memoria de este trabajo fin de grado que tiene por título:

King And Peasants

Lo cual firmo,

Fdo. D. Ismael Carrasco Mkhazni
en la Universidad de Sevilla
02/05/2026

Tu dedicatoria aquí



AGRADECIMIENTOS

No olvides añadir una nota de agradecimiento a quienes hayan contribuido emocionalmente al proyecto fin de Grado.

RESUMEN

Durante la primera década del siglo XXI, los videojuegos de navegador representaron la principal puerta de entrada al ocio digital para una generación de jóvenes, democratizando el entretenimiento gracias a su inmediatez y accesibilidad. Plataformas impulsadas por tecnologías como *Adobe Flash* permitieron que millones de usuarios disfrutaran de experiencias multijugador sencillas y directas, sin barreras de entrada ni hardware costoso. Sin embargo, los hábitos de consumo fueron cambiando progresivamente debido al desplazamiento de la industria hacia grandes producciones y ecosistemas móviles cerrados, culminando con el cese definitivo del soporte de Flash Player en 2020 [1]. El presente Trabajo Fin de Grado, *King and Peasant*, nace de la motivación personal de recuperar esa esencia nostálgica, reconstruyendo la experiencia del juego social de navegador bajo los estándares de la ingeniería de software moderna. El objetivo no es competir con la industria comercial, sino crear un tributo tecnológico que devuelva al usuario la posibilidad de .entrar y jugar con amigos de forma instantánea.

Para conseguir este objetivo, se ha adaptado al entorno web el juego de cartas estratégico asimétrico *King and Peasant* mediante un ciclo de desarrollo ágil basado en Scrum. Dicha implementación se ha llevado a cabo empleando tecnologías modernas como React, Express, Node.js, MariaDB y WebSockets, bajo el patrón arquitectónico Modelo-Vista-Controlador (MVC). Como resultado, se ha desplegado una aplicación web funcional y dockerizada que integra un sistema de gestión de usuarios, salas de espera multijugador (lobbies) y un motor de reglas en tiempo real, demostrando que es posible utilizar las tecnologías web actuales para ofrecer una experiencia interactiva y fluida, apoyada en una base técnica sólida y escalable que cumple con los estándares de la ingeniería del software profesional.

ABSTRACT

During the first decade of the 21st century, browser games represented the main gateway to digital leisure for a generation of young people, democratizing entertainment thanks to their immediacy and accessibility. Platforms driven by technologies such as *Adobe Flash* allowed millions of users to enjoy simple and direct multiplayer experiences, without entry barriers or expensive hardware. However, consumption habits progressively changed due to the industry's shift towards large-scale productions and closed mobile ecosystems, culminating in the definitive end of support for Flash Player in 2020 [1]. The present Bachelor's Thesis, *King and Peasant*, arises from the personal motivation to recover that nostalgic essence, reconstructing the social browser game experience under the standards of modern software engineering. The objective is not to compete with the commercial industry, but to create a technological tribute that gives back to the user the ability to "jump in and play" with friends instantly.

To achieve this goal, the asymmetric strategic card game *King and Peasant* has been adapted to the web environment through an agile development cycle based on Scrum. This implementation has been carried out using modern technologies such as React, Express, Node.js, MariaDB, and WebSockets, under the Model-View-Controller (MVC) architectural pattern. As a result, a functional and dockerized web application has been deployed, integrating a user management system, multiplayer waiting rooms (lobbies), and a real-time rules engine, demonstrating that it is possible to use current web technologies to offer an interactive and fluid experience, supported by a solid and scalable technical foundation that meets the standards of professional software engineering.



ÍNDICE GENERAL

I	Introducción	1
1.	Contexto	3
1.1.	Motivación del Proyecto	4
1.2.	Análisis de viabilidad y acotación del alcance	4
1.3.	Desafíos de ingeniería y retos del desarrollo	5
1.4.	Estructura de la memoria	5
2.	Objetivos	9
2.1.	Definición y delimitación del alcance	10
2.1.1.	Dentro del alcance	10
2.1.2.	Fuera del alcance	10
2.2.	Objetivo General	11
2.3.	Desglose de objetivos técnicos y metodológicos	11
II	Organización del proyecto	13
3.	Estado del Arte y Marco Tecnológico	15
3.1.	Análisis de soluciones existentes y estado del arte	16
3.2.	Arquitectura de comunicación y sincronización	17
3.3.	Tecnologías de <i>Backend</i> y Ecosistema de Desarrollo	18

3.4. Tecnologías de Frontend Reactividad de Interfaz	18
3.5. Gestión de la persistencia de datos	19
3.6. Contenedorización y automatización de pruebas	20
4. Metodología	21
4.1. Estructura organizacional del proyecto	22
4.2. Metodología de desarrollo	22
4.2.1. Análisis de alternativas tradicionales	23
4.2.2. Elección: Metodología Ágil (<i>Scrum</i>)	23
4.3. Adaptación al proyecto: El ciclo de vida ágil	24
4.3.1. Sprints Semanales	24
4.3.2. Coordinación y herramientas de gestión	25
5. Planificación	27
5.1. Resumen temporal del proyecto	28
5.2. Planificación inicial	28
5.2.1. Cronograma de iteraciones (Nivel Micro)	29
5.3. Gestión de desviaciones y medidas correctivas	31
5.3.1. Incidencia técnica: Fallo en el hardware de desarrollo	31
5.3.2. Conciliación académica y personal	31
6. Costes	33
6.1. Resumen de costes del proyecto	34
6.2. Costes de personal	34
6.3. Costes materiales	34
6.4. Costes indirectos	34

III Desarrollo del proyecto	35
7. Análisis y Diseño del Sistema	37
7.1. Análisis de Requisitos	38
7.1.1. Requisitos Funcionales y Actores	38
7.1.2. Requisitos No Funcionales (RNF)	38
7.1.3. Reglas de Negocio y Restricciones del Dominio	39
7.2. Modelado del Sistema	40
7.2.1. Diagrama de Casos de Uso	40
7.2.2. Comportamiento Dinámico: Flujo de Ejecución	41
7.3. Diseño de la Arquitectura y el Sistema	42
7.3.1. Estrategia Organizativa: Monorepositorio	43
7.3.2. Arquitectura del Servidor (Backend)	44
7.3.3. Arquitectura del Cliente (Frontend)	44
7.3.4. Arquitectura de Persistencia Híbrida	45
8. Implementación y Desarrollo del Sistema	49
8.1. Configuración del Entorno y Arquitectura Base	50
8.1.1. Iteración 1: Configuración de Entornos y Metodología Base	51
8.1.2. Iteración 2: Diseño de Interfaces y Orquestación de Datos	53
8.2. Autenticación y Modelado de Datos	54
8.2.1. Iteración 3: Modelado Estructural y Flujo de Negocio	55
8.2.2. Iteración 4: Implementación del Modelo Estructural para los usuarios	56
8.2.3. Iteración 5: Gestión de Identidad, Seguridad y Perfiles	57
8.3. Interacciones Sociales y Comunicación Bidireccional	58
8.3.1. Iteración 6: Interacciones Sociales y Comunicación en Tiempo Real	59

8.4.	Arquitectura del Motor de Juego Asimétrico	61
8.4.1.	Iteración 7: Documentación, Planificación y QA	61
8.4.2.	Iteración 8: Refactorización y Estado Inicial de Partida	62
8.4.3.	Iteración 9: Desarrollo del Motor de Juego y Cartas de Acción	63
8.4.4.	Iteración 10: Condiciones de Victoria y Resolución de Partida	65
8.5.	Aseguramiento de Calidad y Pulido Final	66
8.5.1.	Iteración 11: Interfaces Asíncronas y Modales de Cartas de Acción	67
8.5.2.	Iteración 12: Consolidación Funcional del Motor de Juego y Lógica de Cartas	68
8.5.3.	Iteración 13: Interfaces de Infiltración y Estabilización del Bucle de Partida	70
8.5.4.	Iteración 14: Pulido de Reglas, Corrección de Errores y Refactorización Estructural	71
8.5.5.	Iteración 15: Sistema de Notificaciones Globales y Alertas de Partida	72
8.5.6.	Iteración 16: Automatización de Pruebas y Validación Integral del Sistema	73
9.	Pruebas y Validación del Sistema	77
9.1.	Estrategia de Pruebas y Entorno	77
9.1.1.	Validación del Cliente (Frontend)	77
9.1.2.	Validación del Servidor (Backend)	79
9.1.3.	Validación del Motor Asimétrico (Game Service)	79
9.2.	Integración Continua y Despliegue (CI/CD)	81
IV	Cierre del proyecto	83
10.	Manual de usuario	85

10.1. Índice	86
10.2. Pantalla Inicial	86
10.3. Registro/Inicio de Sesión	87
10.4. Amistad	89
10.5. Listado de Salas	90
10.5.1. Crear Sala	91
10.5.2. Unirse a una Sala:	91
10.5.3. Volver a la sala:	92
10.6. Sala	93
10.7. Partida	93
10.7.1. Interfaz del Juego	93
10.7.2. Acciones de Turno	95
10.8. Fin de la Era y Fin de la Partida	96
11. Conclusiones	97
11.1. Informe post-mortem	98
11.1.1. Lo que ha ido bien	98
11.1.2. Lo que ha ido mal	98
11.1.3. Discusión	98
11.2. Trabajos futuros	98
V Appendices	99
A. Software Product Lines	101
A.1. Software Product Lines	102
A.2. Feature Models	102

A.3. Automated Analysis of Feature Models	104
A.3.1. Scope	104
A.4. Dynamic Software Product Lines (DSPL)	107
A.5. Hypothesis and Objectives	107
Referencias bibliográficas	110

ÍNDICE DE FIGURAS

3.1. Interfaz de <i>Tabletop Simulator</i> , basada en la simulación física tridimensional sin validación de reglas.	16
3.2. Interfaz de <i>Board Game Arena</i> , basada en la automatización del estado de la partida en 2D.	17
7.1. Diagrama General de Casos de Uso del sistema.	41
7.2. Diagrama de Secuencia: Registro, creación de sala y flujo de partida. . .	42
7.3. Diagrama de Arquitectura global del sistema y flujos de comunicación. .	43
7.4. Diagrama Entidad-Relación del sistema.	45
7.5. Arquitectura de seguridad y filtrado de estado mediante el patrón DTO. .	47
8.1. Diagrama de topología de contenedores de la infraestructura base, ilustrando la red interna (bridge) entre el servidor, MariaDB y Redis.	52
8.2. Captura de los mock-ups de la interfaz principal: Tablero de juego asimétrico y salas de espera (Lobbies).	53
10.1. Pantalla Inicio	86
10.2. Pantalla de Registro	87
10.3. Pantalla de Inicio de Sesión	88
10.4. Pantalla de Amistad	89
10.5. Pantalla de Listado de Salas	90
10.6. Pantalla de Listado de Salas: Crear Sala	91
10.7. Pantalla de Listado de Salas: Volver a la Sala	92

10.8. Pantalla de Sala	93
10.9. Pantalla de Partida: Tu Área	94
10.10. Pantalla de Partida: Área del Rival	95
A.1. An example of a Home Integration System	103
A.2. A different view on AAFM distinguishing between information extraction and explanatory operations	105

ÍNDICE DE CUADROS

5.1. Tabla resumen de tiempos y planificación	28
5.2. Cronograma inicial de tareas distribuidas por Sprints semanales.	30
6.1. Tabla resumen de costes	34
8.1. Análisis de valor aportado sobre la arquitectura de comunicación mediante WebSockets.	60
8.2. Descripción algorítmica de la resolución de acciones asíncronas en el servidor.	65
A.1. Most frequently used explanatory operations and their corresponding information extraction operations	109

LISTA DE TAREAS PENDIENTES

■ To Abductive Section in 2.1	102
Figura: A feature model example	103
■ To Abductive Intro	104

PARTE I

INTRODUCCIÓN

CONTEXTO

...

Ernest Hemingway (1899–1961),
Novelist

***E**l siguiente capítulo aborda la concepción y el marco operativo del proyecto King and Peasant. Se expone la motivación inicial centrada en el desarrollo colaborativo y el interés por la arquitectura de videojuegos. Asimismo, se justifica la acotación del alcance del trabajo basada en un análisis de viabilidad técnica y temporal, fundamentando la decisión de adaptar un juego de mesa preexistente mediante tecnologías web modernas. Por último, se detallan los principales desafíos de ingeniería identificados, estableciendo la base para los objetivos y el desarrollo técnico que se describen en los capítulos posteriores.*

El desarrollo de sistemas interactivos multijugador representa uno de los retos más dinámicos y exigentes dentro de la ingeniería de software contemporánea. Este proyecto, centrado en la adaptación digital del juego de mesa *King and Peasant*, surge como una respuesta técnica a la voluntad de crear experiencias de ocio accesibles y sociales bajo estándares profesionales. Lejos de ser un mero ejercicio de programación, el trabajo se concibe como un proyecto integral de ingeniería donde la gestión de recursos, la toma de decisiones estratégicas y la coordinación de un equipo de desarrollo son tan críticas como la implementación del código.

A continuación, se detalla el contexto que da origen a este Trabajo Fin de Grado (en adelante, TFG). Se comienza explorando las motivaciones que impulsan el desarrollo colaborativo, para posteriormente analizar la viabilidad del proyecto y los criterios técnicos que llevaron a definir su alcance actual. Finalmente, se presentan los desafíos de ingeniería que componen la solución propuesta, centrados en la transición del entorno físico al digital y la gestión de la concurrencia en tiempo real.

1.1 MOTIVACIÓN DEL PROYECTO

La motivación principal detrás de la elección de este TFG es el interés por la arquitectura de videojuegos y el deseo de invertir el esfuerzo académico en un producto creativo y estimulante, resultando así intrínsecamente gratificante durante su ciclo de construcción. A su vez, dado el carácter aislado que suelen tener los proyectos académicos, se tomó la decisión de realizar el desarrollo como un esfuerzo mediante un equipo compuesto por dos personas. Al plantearse como un trabajo colaborativo, el proyecto se convierte en un ejercicio de coordinación técnica y trabajo en equipo, simulando las dinámicas y exigencias de un entorno profesional real en la industria del software.

1.2 ANÁLISIS DE VIABILIDAD Y ACOTACIÓN DEL ALCANCE

El planteamiento inicial de diseñar y desarrollar íntegramente un videojuego original fue descartado en las fases preliminares. La inversión en horas del diseño desde cero de mecánicas, el balanceo de reglas y la curva de aprendizaje presente en los motores gráficos actuales excedía el alcance de un TFG. Dado que la viabilidad de un proyecto de ingeniería de software depende de la correcta alineación entre los objetivos y los recursos disponibles, se tomó la decisión de acotar el alcance funcional adoptando el motor de reglas de un juego de mesa físico ya existente, además de hacer uso de los

conocimientos adquiridos a lo largo de la carrera, en tecnologías de desarrollo web.

En consecuencia, se optó por la adaptación digital de *King and Peasant*, un juego de mesa de tipo *Print & Play* con reglas ya consolidadas creado por la empresa nPipsGames [12]. Esta decisión permitió desplazar el foco hacia la resolución de los retos tecnológicos de la implementación: la construcción de una arquitectura robusta, la gestión de la concurrencia en tiempo real y la coordinación de un equipo de desarrollo colaborativo utilizando el stack tecnológico ya dominado por los autores (React y Node.js)

1.3 DESAFÍOS DE INGENIERÍA Y RETOS DEL DESARROLLO

El desarrollo de este proyecto presenta tres desafíos técnicos y organizativos que justifican su complejidad. En primer lugar, la adaptación de un juego de mesa como *King and Peasant* conlleva una labor de abstracción de reglas. No solo se trata de digitalizar una interfaz, sino de transformar las mecánicas pensadas para la interpretación humana y física para que se integren en un sistema de estados robusto con la capacidad de validar acciones y prevenir comportamientos no permitidos de forma automatizada.

En segundo lugar, el componente multijugador añade el reto de la concurrencia y la sincronización de estados. Es necesario asegurar que todos los jugadores dispongan de la misma información de manera instantánea, al mismo tiempo que se garantiza el correcto manejo de múltiples eventos simultáneos, manteniendo la integridad del estado global de la partida.

Finalmente, la coordinación de un equipo de desarrollo supone un desafío organizativo adicional de colaboración. La gestión de un repositorio compartido, el uso de ramas de trabajo y la integración de módulos desarrollados en paralelo exigen un uso constante de metodologías de trabajo ágiles y herramientas de control de versiones para asegurar la coherencia del producto final.

1.4 ESTRUCTURA DE LA MEMORIA

El presente documento se ha estructurado de manera secuencial para guiar al lector a través de todo el ciclo de vida de la ingeniería del software aplicada al proyecto *King and Peasant*. La memoria se divide en los siguientes capítulos:

- **Capítulo 1. Introducción:** Presenta el contexto del proyecto, la motivación principal que ha impulsado su desarrollo, así como los objetivos generales y específicos que

se pretenden alcanzar.	1
■ Capítulo 2. Estado del arte: Analiza el panorama histórico y actual de los videojuegos de navegador, explorando las referencias lúdicas y la teoría detrás de los juegos de diseño asimétrico que sirven de base para este proyecto.	2 3 4
■ Capítulo 3. Entorno Tecnológico: Justifica de manera técnica y arquitectónica la selección de las herramientas y lenguajes que componen el <i>stack</i> de desarrollo (React, Node.js, Prisma, MariaDB, Redis, Socket.io y Docker).	5 6 7
■ Capítulo 4. Metodología de trabajo: Describe las prácticas organizativas adoptadas por el equipo, detallando la adaptación de metodologías ágiles (Scrum) y las políticas de control de versiones empleadas en el repositorio.	8 9 10
■ Capítulo 5. Planificación: Desglosa la división del tiempo en iteraciones o <i>sprints</i> , mostrando el cronograma de ejecución real y las desviaciones temporales afrontadas durante el desarrollo.	11 12 13
■ Capítulo 6. Costes: Realiza un análisis de viabilidad económica, cuantificando los gastos de personal, licencias y amortización de hardware y software necesarios para llevar a cabo el proyecto.	14 15 16
■ Capítulo 7. Requisitos y Diseño del Sistema: Establece los cimientos funcionales del juego. Define los requisitos, las reglas de negocio (RN) inquebrantables, el modelo de datos Entidad-Relación y la arquitectura de la información mediante diagramas UML.	17 18 19 20
■ Capítulo 8. Implementación y Desarrollo del Sistema: Documenta la evolución técnica del software a lo largo de 16 iteraciones. Explica la resolución algorítmica de mecánicas complejas, el diseño de la comunicación en tiempo real y la refactorización arquitectónica.	21 22 23 24
■ Capítulo 9. Pruebas y Validación del Sistema: Expone la estrategia de aseguramiento de calidad (QA) implementada, abarcando desde las pruebas de integración en el <i>backend</i> hasta la orquestación del despliegue continuo (CI/CD).	25 26 27
■ Capítulo 10. Manual de Usuario: Proporciona una guía visual y funcional sobre cómo interactuar con la plataforma final, ilustrando los flujos de registro, gestión social y el desarrollo de una partida.	28 29 30

- 1 ■ **Capítulo 11. Conclusiones y Trabajo Futuro:** Hace un balance crítico sobre el
- 2 grado de cumplimiento de los objetivos iniciales, reflexiona sobre las lecciones
- 3 aprendidas y propone posibles vías de ampliación para el ecosistema del juego.

OBJETIVOS

The good parts of a book may be only something a writer is lucky enough to overhear or it may be the wreck of his whole damn life – and one is as good as the other.

*Ernest Hemingway (1899–1961),
Novelist*

Este capítulo establece el marco operativo y las metas fundamentales del TFG. En primer lugar, se delimita el alcance del proyecto, definiendo las responsabilidades técnicas asumidas y las restricciones funcionales del sistema. A continuación, se formula el objetivo general centrado en la adaptación digital de King and Peasant, para finalmente desglosarlo en un conjunto de objetivos específicos técnicos, de diseño y metodológicos que servirán como hitos medibles a lo largo del ciclo de desarrollo.

Tras establecer el contexto histórico y la motivación del proyecto en el primer capítulo, resulta imprescindible trazar una hoja de ruta técnica clara. Este segundo capítulo tiene como propósito fundamental acotar la magnitud del desarrollo y definir con precisión las metas del trabajo. Para ello, se comenzará delimitando el alcance funcional del sistema, diferenciando las competencias estrictamente ligadas a la ingeniería de software de aquellos elementos descartados por viabilidad temporal. Posteriormente, se formulará el objetivo general que compone la digitalización de *King and Peasant*, desglosándolo finalmente en una serie de hitos técnicos y metodológicos que guiarán las futuras fases de implementación y despliegue.

2.1 DEFINICIÓN Y DELIMITACIÓN DEL ALCANCE

La correcta delimitación del alcance es un factor crítico para asegurar la viabilidad del TFG. Dadas las limitaciones expuestas en el capítulo anterior, es necesario establecer ciertas fronteras en el desarrollo del proyecto. A continuación, se detallan los elementos que componen el alcance y aquellos que han sido explícitamente excluidos.

2.1.1 Dentro del alcance

El desarrollo abarca el ciclo de vida de la aplicación web, incluyendo:

- **Arquitectura de comunicación:** Implementación de un servidor capaz de gestionar sesiones concurrentes y comunicación en tiempo real mediante *WebSockets*.
- **Motor de reglas digital:** Traducción del reglamento de *King and Peasant* a una lógica de negocio que valide automáticamente los estados de la partida.
- **Gestión de usuarios:** Diseño de persistencia de datos para perfiles, amistades e histórico de partidas.
- **Despliegue:** Contenedorización del entorno de producción mediante *Docker* para garantizar su accesibilidad.

2.1.2 Fuera del alcance

Para centrar los esfuerzos en la calidad técnica del software, quedan excluidos:

- **Diseño original de mecánicas:** Se delega el balanceo y autoría del juego en el reglamento físico creado por nPipsGames.

- **Desarrollo móvil nativo:** El sistema se limita al entorno web, sin contemplar aplicaciones específicas para iOS o Android.
- **Monetización:** El producto se concibe como una herramienta de ocio gratuita, excluyendo pasarelas de pago o sistemas publicitarios.
- **Sistema competitivo (*Ranked*):** No se implementarán sistemas de emparejamiento basados en habilidad (*matchmaking*) ni puntuaciones de clasificación global (tipo Elo).
- **Economía virtual:** Queda excluida la creación de divisas internas (*in-game currency*) y sistemas de progresión basados en la adquisición de bienes virtuales.
- **Inteligencia Artificial (PvE):** El proyecto se centra estrictamente en la interacción jugador contra jugador (PvP), por lo que se descarta el desarrollo de agentes autónomos o *bots* contra los que jugar en solitario.

2.2 OBJETIVO GENERAL

El objetivo central de este TFG es el diseño, implementación y despliegue de la versión digital del juego asimétrico *King and Peasant* en una plataforma web multijugador. La meta final es ofrecer un ecosistema persistente que garantice la inmediatez de los juegos de navegador clásicos mediante el uso de tecnologías web modernas, una arquitectura cliente-servidor robusta y metodologías profesionales de desarrollo, asegurando una experiencia de usuario fluida, sincronizada y de alta disponibilidad.

2.3 DESGLOSE DE OBJETIVOS TÉCNICOS Y METODOLÓGICOS

Para alcanzar la meta global descrita anteriormente, el proyecto se desglosa en los siguientes objetivos específicos, los cuales actúan como hitos medibles del desarrollo:

- **Objetivo 1. Desarrollo de la lógica de juego y sincronización en tiempo real.**
Implementar el motor de reglas del juego de mesa asegurando la integridad de la partida. Esto implica el uso de *WebSockets* para garantizar una comunicación bidireccional instantánea entre el servidor y los clientes, de modo que todos los jugadores perciban el mismo estado del tablero simultáneamente y sin latencia perceptible.

- **Objetivo 2. Construcción de un ecosistema social persistente.** 1
 Desarrollar un sistema de perfiles de usuario robusto que vaya más allá de la 2
 partida. Esto incluye el registro y autenticación segura de usuarios, gestión de 3
 listas de amigos, historial de partidas y estadísticas de rendimiento, fomentando 4
 la retención y la comunidad dentro de la plataforma. 5

- **Objetivo 3. Diseño de una Experiencia de Usuario (UX/UI) integral e intuitiva.** 6
 Implementar una interfaz gráfica limpia y ergonómica que traslade fielmente la 7
 fluidez del juego de mesa original al entorno digital. Este diseño debe garantizar 8
 una navegación coherente y accesible no solo durante el transcurso de la partida 9
 en el tablero, sino en todo el ecosistema de la plataforma, abarcando la gestión de 10
 perfiles, la autenticación y la administración de salas de espera. 11

- **Objetivo 4. Gestión integral del ciclo de vida del proyecto.** 12
 Al tratarse de un desarrollo colaborativo realizado por un equipo de dos personas, 13
 un objetivo transversal es la aplicación de metodologías ágiles y buenas prácticas 14
 de ingeniería. Esto abarca el control de versiones (*Git*), la integración continua, 15
 el reparto de tareas mediante tableros *Kanban* y la coordinación eficiente entre el 16
Frontend y el *Backend*. 17

- **Objetivo 5. Despliegue y accesibilidad.** 18
 Asegurar que el resultado final sea accesible públicamente a través de internet, con- 19
 figurando un entorno de producción real que soporte la concurrencia de usuarios 20
 y demuestre la viabilidad técnica de la solución propuesta. 21

PARTE II

ORGANIZACIÓN DEL PROYECTO

ESTADO DEL ARTE Y MARCO TECNOLÓGICO

The good parts of a book may be only something a writer is lucky enough to overhear or it may be the wreck of his whole damn life – and one is as good as the other.

*Ernest Hemingway (1899–1961),
Novelist*

En este capítulo presenta el estado del arte y el marco tecnológico que sustenta el desarrollo de King and Peasant. En primer lugar, se analizan las soluciones existentes en la digitalización de juegos de mesa, justificando la adopción de un modelo basado en la automatización de reglas frente a la simulación física. A continuación, se detalla y justifica la elección del stack tecnológico, abordando la arquitectura de comunicación mediante WebSockets, el entorno de ejecución asíncrono en Node.js, el renderizado reactivo con React y un sistema de persistencia dual basado en MariaDB y Redis. Finalmente, se expone la estrategia de contenedorización con Docker y la política de pruebas automatizadas para garantizar la calidad y portabilidad del sistema.

3.1 ANÁLISIS DE SOLUCIONES EXISTENTES Y ESTADO DEL ARTE

1

En la actualidad existen dos acercamientos a la hora de digitalizar un juego de mesa contemporáneo como lo es *King and Peasant*. Por un lado, tenemos el ejemplo de *Tabletop Simulator* (véase la Figura §3.1), que se basa en un entorno de simulación física operando como un "cajón de arena" (*sandbox*) [15]. El cumplimiento de las reglas se delega íntegramente a los usuarios, careciendo de una validación lógica que incremente la carga cognitiva del usuario y ralentiza el flujo de la partida.

2

3

4

5

6

7

Por otro lado, se encuentra *Board Game Arena* (véase la Figura §3.2), que está basado en un motor de validación de estados. En este modelo el cumplimiento de las reglas y las transiciones de estado se delegan al sistema, actuando así como un árbitro imparcial dentro de la partida garantizando la integridad de las mecánicas [6]. El presente TFG se alinea con este segundo enfoque, ya que lo que se busca es crear un motor de reglas estricto que ofrezca una experiencia de juego automatizada, accesible desde el navegador y libre de la fricción que suponen los simuladores tridimensionales

8

9

10

11

12

13

14



Figura 3.1: Interfaz de *Tabletop Simulator*, basada en la simulación física tridimensional sin validación de reglas.



Figura 3.2: Interfaz de *Board Game Arena*, basada en la automatización del estado de la partida en 2D.

3.2 ARQUITECTURA DE COMUNICACIÓN Y SINCRONIZACIÓN

Dada la naturaleza interactiva de un juego multijugador, la sincronización en tiempo real entre múltiples clientes es un punto crítico dentro del desarrollo de *King and Peasant*. Para este propósito, implementar el motor del juego sobre una arquitectura web tradicional resultaba inviable debido a las limitaciones estructurales del protocolo *HTTP*. El uso de técnicas como el *Long-Polling (polling)*, donde el cliente interroga constantemente al servidor en busca de actualizaciones, introduciría una sobrecarga significativa en la red debido al envío repetitivo de cabeceras *HTTP*, generando latencias perceptibles que degradan la experiencia de juego [9].

Como alternativa a una API *REST*, se decidió utilizar el protocolo *WebSocket* como núcleo de las comunicaciones de la partida. Como define el estándar oficial de la IETF [8], esta tecnología proporciona un canal de comunicación bidireccional sobre una única conexión *TCP* persistente. Esto permite el envío de eventos asíncronos (*server-push*) en el mismo instante en el que se modifica el estado, validando las jugadas y actualizando instantáneamente la interfaz de todos los clientes conectados, reduciendo a su vez el coste computacional y garantizando una sincronización fluida.

3.3 TECNOLOGÍAS DE *BACKEND* Y ECOSISTEMA DE DESARROLLO

El alto volumen de conexiones concurrentes que presenta un juego multijugador exige un entorno de ejecución capaz de gestionar con eficiencia el consumo elevado de recursos que se produce al mantener las conexiones abiertas en un estado de sincronización permanente. En el caso de los servidores web tradicionales, el modelo de concurrencia basado en hilos (*thread-per-request*) se ve afectado por el cambio de contexto (*context switching*) [13], provocando un rápido agotamiento de la memoria y cuellos de botella. Por otro lado, la mantenibilidad del código y el uso de lenguajes unificados constituyó un factor determinante a la hora de decantarse por una arquitectura de servidor.

Por estos motivos, se adoptó *Node.js*. Su arquitectura basada en un bucle de eventos (*Event Loop*) procesa las solicitudes de red de forma asíncrona, evitando bloquear el hilo principal durante la espera de respuestas de la base de datos o de los clientes. Este diseño arquitectónico permite al sistema mantener activas miles de conexiones bidireccionales de manera simultánea con un consumo residual de recursos. Además, la capacidad de usar lenguajes isomórficos reduce la sobrecarga cognitiva del equipo de desarrollo y permite la compartición directa de módulos lógicos entre cliente y servidor [4].

3.4 TECNOLOGÍAS DE FRONTEND REACTIVIDAD DE INTERFAZ

La implementación de un juego de mesa en un entorno digital multijugador requiere elementos como un tablero interactivo, el cual, en términos de ingeniería, no es más que la proyección visual de una compleja máquina de estados. Sin embargo, se plantea una serie de retos técnicos a la hora de diseñar este componente.

En primer lugar, la manipulación directa del *Document Object Model* (DOM) para actualizar la interfaz resulta computacionalmente ineficiente. El navegador se ve obligado a recalcular estilos y repintar la pantalla cada vez que se produce una modificación, lo que provoca que la fluidez de la experiencia se vea degradada [10]. En segundo lugar, abordar una interfaz que cuenta con tableros, mazos, perfiles y salas de espera bajo una arquitectura monolítica dificultaría enormemente su mantenibilidad y fomentaría la duplicación de código. Finalmente, se le suma el riesgo de desincronización de los estados visuales, resultando en validaciones del *backend* que no se reflejan en la información gráfica [2].

Como solución a todos estos problemas, la arquitectura del cliente se construyó utilizando *React*, una librería que impone un paradigma de desarrollo declarativo y

basado en componentes. En primer lugar, gracias a la implementación de un *Virtual DOM*, la ineficiencia computacional se mitiga. Al recibir un nuevo evento, *React* genera una representación en memoria de la interfaz y la compara con la actual mediante un algoritmo de diferenciación (*diffing*), aplicando al DOM real únicamente las modificaciones estrictamente necesarias. En segundo lugar, su arquitectura modular minimiza el riesgo que supone construir un sistema monolítico, permitiendo dividir la interfaz en unidades lógicas aisladas e independientes, facilitando la encapsulación y garantizando la mantenibilidad a largo plazo del código. Por último, el paradigma declarativo de *React* permite concibir la interfaz matemáticamente como una función del estado subyacente ($UI = f(estado)$) [2], convirtiendo el tablero en un reflejo exacto de la información de la partida y las reglas dictaminadas por el *backend* [10].

3.5 GESTIÓN DE LA PERSISTENCIA DE DATOS

Dentro de la información gestionada por el ecosistema encontramos requisitos técnicos independientes, lo que implica la necesidad de implementar un enfoque de persistencia adecuado para cada uno de ellos fundamentado en su naturaleza. Por un lado, la información estructural, como pueden ser las credenciales, perfiles de usuario, historial de partidas o la gestión de salas de espera (*lobbies*), exige un almacenamiento persistente y estructurado que garantice las propiedades ACID (Atomicidad, Consistencia, Aislamiento y Durabilidad) [7]. Para satisfacer esta necesidad, y tras evaluar alternativas relacionales robustas como *PostgreSQL*, se seleccionó el motor de bases de datos relacional *MariaDB* por su agilidad en consultas y su perfecta integración con el ecosistema de *Node.js*. Este motor se encarga de gestionar de forma centralizada la información de los usuarios y la configuración inicial de las partidas. La interacción con este motor relacional se abstraigo mediante *Prisma ORM*, una decisión que previene vulnerabilidades de inyección SQL y garantiza un tipado estricto entre el esquema de la base de datos y la lógica del *backend*.

Por otro lado, el estado del tablero interactivo se convierte en un dato efímero sometido a un volumen masivo de operaciones de lectura y escritura concurrentes. Hacer uso de una base de datos relacional para registrar las modificaciones generaría un cuello de botella debido a la latencia de entrada/salida (I/O), ralentizando la comunicación. Para mitigar este problema, se implementó *Redis* como almacén de estructuras de datos en memoria (*In-Memory Data Grid*) [5]. A diferencia de sistemas de caché simples como *Memcached*, *Redis* soporta el manejo nativo de estructuras de datos complejas, una característica indispensable para modelar el estado de un juego. Hacer uso de la

memoria RAM para almacenar el estado dinámico de la partida permite que el servidor valide y comparta el resultado de las jugadas con tiempos de respuesta mucho menores, delegando en *MariaDB* la persistencia del resultado final una vez concluido el encuentro.

3.6 CONTENEDORIZACIÓN Y AUTOMATIZACIÓN DE PRUEBAS

El complejo funcionamiento intrínseco del motor de reglas asimétrico requería mecanismos rigurosos para garantizar la calidad y la estabilidad del software. El mal funcionamiento de las validaciones de estado o de las transiciones de la partida degradaría la experiencia de juego. En respuesta, haciendo uso de los marcos de trabajo *Jest* y *Vitest*, se implementó una batería de pruebas automatizadas que aseguraban el correcto comportamiento del motor de validación ante cualquier escenario [11]. Dichas herramientas han permitido aislar y testear unitariamente la lógica del *backend* y los componentes del *frontend*.

De forma paralela, la gestión de la infraestructura se apoyó en la tecnología de contenedores *Docker* [14]. El principal problema que se buscó mitigar con esta adopción fue la inconsistencia de entornos al tratarse de un desarrollo colaborativo entre dos personas. Resultaba necesario aislar los servicios para evitar fallos derivados de las diferencias de *hardware* o configuración del sistema operativo. Por este motivo, se estableció una arquitectura de ejecución híbrida. Por un lado, los servicios de bases de datos (*MariaDB* y *Redis*) se ejecutan exclusivamente mediante contenedores, garantizando que el equipo opere siempre sobre versiones idénticas de estos sistemas. Por otro lado, la capa de aplicación (*frontend* y *backend*) se implementó con un modelo flexible que permite su lanzamiento en local para agilizar la iteración de código, o bien su ejecución contenedorizada. A su vez, esta estandarización de los servicios resulta idónea para simplificar y asegurar el futuro despliegue de la plataforma en entornos de producción.

METODOLOGÍA

The good parts of a book may be only something a writer is lucky enough to overhear or it may be the wreck of his whole damn life – and one is as good as the other.

*Ernest Hemingway (1899–1961),
Novelist*

***E**n este capítulo se define el marco de trabajo y los procedimientos organizativos adoptados para la ejecución del proyecto. Se analiza la estructura del equipo de desarrollo y se realiza un estudio comparativo de distintas metodologías de ingeniería del software, justificando la elección de un enfoque ágil frente a modelos tradicionales. Finalmente, se detalla la adaptación de Scrum a un equipo reducido para garantizar un desarrollo iterativo y eficiente.*

4.1 ESTRUCTURA ORGANIZACIONAL DEL PROYECTO

1

El desarrollo de *King and Peasant* ha sido llevado a cabo por un equipo de trabajo constituido por dos ingenieros. Dada la naturaleza reducida del grupo y la fuerte interdependencia técnica entre los componentes del sistema (cliente y servidor), se descartaron las jerarquías verticales en favor de una estructura horizontal y colaborativa.

En lugar de segregar rígidamente los roles, delegando la capa de *Frontend* exclusivamente a un integrante y el *Backend* a otro, ambos miembros del equipo han asumido responsabilidades como desarrolladores *Full Stack*.

Esta decisión arquitectónica a nivel de equipo ha reportado dos beneficios fundamentales para el ciclo de vida del proyecto:

- **Flexibilidad operativa:** Ambos integrantes poseen una visión global y profunda de la arquitectura. Esto ha permitido mitigar cuellos de botella operativos; si un desarrollador no estaba disponible o se encontraba bloqueado en una tarea compleja, el otro poseía el contexto técnico necesario para asistir o continuar el desarrollo sin retrasos.
- **Revisión de código cruzada:** Al conocer ambos la tecnología empleada en todas las capas del sistema, el proceso de validación por pares (*Code Review*) ha resultado mucho más fluido y riguroso. Esta auditoría mutua constante ha sido clave para reducir la introducción de deuda técnica temprana.

A nivel de gestión diaria, la responsabilidad se distribuyó mediante un reparto de tareas dinámico. Al inicio de cada ciclo de trabajo, se evaluaba la complejidad de los requisitos y se asignaban las funcionalidades específicas, garantizando en todo momento un equilibrio equitativo en la carga de trabajo de ambos ingenieros.

4.2 METODOLOGÍA DE DESARROLLO

24

Para determinar el modelo de ciclo de vida más adecuado para la naturaleza de este proyecto, se realizó un análisis previo de las metodologías de ingeniería del software más consolidadas en la industria. A continuación, se exponen las alternativas evaluadas y la justificación técnica de la elección final.

4.2.1 Análisis de alternativas tradicionales

Modelo en Cascada (*Waterfall*)

Este enfoque predictivo clásico propone una secuencia puramente lineal de fases (Requisitos, Análisis, Diseño, Implementación, Pruebas y Mantenimiento). Bajo este paradigma, cada etapa debe finalizarse antes de poder comenzar la siguiente.

Motivo del descarte: El desarrollo de un videojuego conlleva una incertidumbre inherente respecto a la jugabilidad y la experiencia final del usuario. El modelo en cascada resulta excesivamente rígido ante esta realidad; si durante la fase de pruebas se detecta que una mecánica de juego presenta fallos de diseño o no resulta atractiva, el coste temporal de retroceder a la fase de análisis para rediseñarla resulta inasumible. Este proyecto requería de una estructura que permitiera pivotar y ajustar requisitos sobre la marcha basándose en el *feedback* interactivo.

Modelo en Espiral

Este modelo evolutivo se centra en la evaluación continua y la mitigación de riesgos mediante iteraciones concéntricas que combinan planificación, análisis de viabilidad y prototipado.

Motivo del descarte: Aunque este paradigma soluciona la rigidez del modelo en cascada, el modelo en espiral introduce una carga de gestión burocrática y de documentación (*overhead*) desproporcionada para un equipo compuesto por solo dos personas. El tiempo invertido en la evaluación formal de riesgos en cada iteración restaría demasiados recursos a la fase de implementación efectiva del código.

4.2.2 Elección: Metodología Ágil (*Scrum*)

Tras descartar los modelos predictivos y evolutivos pesados, la organización del proyecto se decantó por la adopción de una metodología ágil fundamentada en el marco de trabajo *Scrum*. Este enfoque abraza la incertidumbre mediante un desarrollo iterativo e incremental, donde el producto funcional se construye gradualmente en ciclos de tiempo encapsulados.

Esta elección metodológica se justifica por dos pilares clave del proyecto:

1. **Entrega temprana de valor:** Trabajar mediante incrementos funcionales permite consolidar una visión clara del estado del producto desde las primeras semanas

de desarrollo. Esto resulta vital en arquitecturas complejas, ya que permite validar técnicamente la integración entre el cliente y el servidor sin tener que esperar al final del proyecto.

2. **Adaptabilidad y tolerancia al cambio:** Facilita enormemente la comprobación de las reglas del juego, la reestructuración de la interfaz o la alteración de mecánicas tras las pruebas iniciales, todo ello sin desestabilizar la planificación global.

4.3 ADAPTACIÓN AL PROYECTO: EL CICLO DE VIDA ÁGIL

Si bien el marco de trabajo *Scrum* define roles (como el *Scrum Master* o el *Product Owner*) y herramientas diseñadas para equipos de mayor tamaño, en este proyecto se ha ejecutado una adaptación del modelo orientada a la eficiencia. Se ha prescindido de la rigidez burocrática para maximizar la agilidad de un micro-equipo colaborativo.

4.3.1 Sprints Semanales

El esfuerzo de desarrollo se ha dividido en iteraciones temporales cortas o *Sprints* con una duración estricta de una semana. Esta cadencia temporal, notablemente más rápida que las iteraciones de quince días habituales en la industria, ha permitido un nivel de control granular y un seguimiento muy preciso del progreso frente al calendario académico.

El ciclo de cada iteración se ha estructurado en tres eventos fundamentales:

- ***Sprint Planning* (Planificación):** Al inicio de la semana de trabajo, el equipo evalúa y selecciona las tareas prioritarias del *Product Backlog* (pila de producto), estableciendo el objetivo técnico al que se comprometen para esa iteración.
- **Ejecución en paralelo:** Desarrollo efectivo de las funcionalidades y mecánicas acordadas.
- ***Sprint Review* (Revisión e Integración):** Esta validación se lleva a cabo mediante un proceso de revisión cruzada (*Peer Review*), donde cada desarrollador audita el código escrito por el otro antes de autorizar su integración definitiva en la rama principal del repositorio.

1 4.3.2 Coordinación y herramientas de gestión

2 Para soportar y dar viabilidad a esta metodología iterativa, el equipo ha empleado
3 herramientas estándar de la industria, asegurando una comunicación constante y la
4 trazabilidad del código:

- 5 ■ **Reuniones de sincronización (*Daily Stand-up*):** Encuentros breves y altamente
6 focalizados (no superiores a 15 minutos) diseñados para coordinar el trabajo de la
7 jornada y, sobre todo, para identificar y mitigar posibles bloqueos técnicos entre
8 ambos desarrolladores.
- 9 ■ **Tablero *Kanban*:** Para la gestión del flujo de trabajo se ha utilizado la herramienta
10 *GitHub Projects*. Este panel permite visualizar en tiempo real el estado evolutivo de
11 las tareas mediante columnas de progreso (“Pendiente”, “En curso”, “En Revisión”,
12 “Terminado”).
- 13 ■ **Estrategia de ramificación:** Uso estricto de *Git* para gestionar el ciclo de vida
14 del código fuente. Se ha adoptado una estrategia basada en la creación de ramas
15 independientes por cada nueva funcionalidad (*Feature Branches*), lo cual resulta
16 imprescindible para que dos desarrolladores modifiquen la misma base de código
17 sin generar conflictos críticos de integración.

PLANIFICACIÓN

The good parts of a book may be only something a writer is lucky enough to overhear or it may be the wreck of his whole damn life – and one is as good as the other.

*Ernest Hemingway (1899–1961),
Novelist*

***E**n este capítulo se expone la planificación temporal del proyecto, estructurada mediante la metodología ágil definida previamente. Se detalla el cronograma de ejecución a través de hitos mensuales y Sprints semanales, abarcando desde la configuración inicial de la arquitectura hasta el despliegue final del sistema. Asimismo, se documentan las desviaciones surgidas durante el ciclo de vida del desarrollo y se analizan las medidas correctivas adoptadas por el equipo para garantizar el cumplimiento de los plazos.*

5.1 RESUMEN TEMPORAL DEL PROYECTO

1

A nivel global, el proyecto abarca un periodo de desarrollo de cinco meses, desde la configuración inicial de la arquitectura hasta la redacción y entrega de la presente memoria técnica. En la Tabla §5.1 se exponen los parámetros temporales fundamentales.

2
3
4

Resumen del proyecto	
Fecha de inicio	05/01/2026
Fecha de fin	15/05/2026
Periodicidad de las revisiones	1 semanas (Sprints)
Carga de trabajo semanal	12 horas
Horas totales previstas	300 horas
Horas finales	240 horas

Cuadro 5.1: Tabla resumen de tiempos y planificación

5.2 PLANIFICACIÓN INICIAL

5

Dada la adopción de un *stack* tecnológico complejo y la interdependencia entre el cliente y el servidor, el calendario general se planteó como una hoja de ruta dividida en grandes hitos mensuales. Esta estructura permitió fijar objetivos claros para cada fase del ciclo de vida:

6
7
8
9

- **Enero - Arquitectura e Infraestructura:** Configuración inicial del repositorio, contenedorización de los servicios mediante *Docker*, implementación del sistema de autenticación de usuarios y salas de espera (*lobbies*), e inicialización del núcleo de la aplicación.
- **Febrero - Core del Juego:** Desarrollo íntegro de la lógica de la partida, diseño del tablero interactivo y aseguramiento de la sincronización bidireccional en tiempo real.
- **Marzo - Reglas avanzadas y Seguridad:** Implementación de las mecánicas asimétricas complejas, pulido de la validación de estados en el *backend* y blindaje de la seguridad de la plataforma.
- **Abril - Funcionalidades extendidas y Testing:** Integración de características periféricas (como sistemas de interacción social o estadísticas), ejecución exhaustiva

10
11
12
13
14
15
16
17
18
19
20
21

de las baterías de pruebas automatizadas y aseguramiento de la calidad del código.

- **Mayo - Despliegue y Cierre:** Finalización de la presente memoria técnica, despliegue final del sistema y preparación para la defensa académica.

La determinación de estas fechas no ha sido arbitraria, sino que responde a un análisis de la carga lectiva y los hitos del calendario académico de la *Escuela Técnica Superior de Ingeniería Informática*. Se estableció un ritmo de trabajo sostenido y diario con el objetivo de compatibilizar el desarrollo del proyecto con el resto de asignaturas del cuarto curso. Esta cadencia diaria ha sido la herramienta principal para evitar la acumulación de tareas en periodos de exámenes, permitiendo que el avance del proyecto fuera independiente de las fluctuaciones de carga de trabajo de otras materias.

5.2.1 Cronograma de iteraciones (Nivel Micro)

Para materializar los hitos mensuales, el trabajo se fragmentó en *Sprints* semanales. En la Tabla §5.2 se detalla la planificación inicial extraída del tablero Kanban, mostrando los objetivos técnicos asignados a cada periodo. Cabe destacar que todas las iteraciones incluyen de forma implícita tareas de documentación continua y reuniones de *Sprint Review & Planning*.

Mes	Semana (Sprint)	Objetivos Técnicos Principales
Enero	12 - 18 Ene	Inicialización de repositorio, Set Up Docker, Frontend/Backend y bases de datos. Diagramas UML.
	19 - 25 Ene	Set Up Despliegue Continuo, Tests, Metodología Git y Mock-ups UI.
	26 - 1 Feb	Desarrollo de Pantalla de Inicio y Login/Registro.
Febrero	2 - 8 Feb	Pantalla de Listado de Salas.
	9 - 15 Feb	Pantalla de Usuario (Editar), Sistema de Amistad y Pantalla de Sala.
	16 - 22 Feb	Funcionalidad invitar usuario, Pantalla Partida Prueba y Chat de Partida.
	23 - 1 Mar	Sistema de Barajas/Cartas, Inspección de roles y Flujo de estados/turnos.
Marzo	2 - 8 Mar	Accionar cartas, Temporizador, UI extra de partida y Acciones Simples.
	9 - 15 Mar	Set up inicio partida, Fin de Era/Partida y Condiciones finales.
	16 - 22 Mar	Implementación de Acciones Complejas 1.
	23 - 29 Mar	Implementación de Acciones Complejas 2 y Pantalla de Ganador.
Abril	30 - 5 Abr	Mover cartas (<i>Drag & Drop</i>) y Animaciones de cartas.
	6 - 12 Abr	Test Lógica de Juego 1, Test Sala y Test Usuarios.
	13 - 19 Abr	Test Lógica de Juego 2, Rediseño estético final.
	20 - 3 May	Refactorización general y solución de errores (<i>Bugfixing</i>).
Mayo	4 - 10 May	Pruebas finales e integración completa.
	11 - 15 May	Finalización y revisión de la memoria técnica.

Cuadro 5.2: Cronograma inicial de tareas distribuidas por Sprints semanales.

5.3 GESTIÓN DE DESVIACIONES Y MEDIDAS CORRECTIVAS

A pesar de la planificación inicial, el desarrollo de un proyecto de ingeniería está sujeto a imprevistos técnicos y personales. Siguiendo las directrices de una metodología ágil, el equipo ha demostrado capacidad de adaptación mediante la aplicación de medidas correctivas inmediatas ante las desviaciones ocurridas.

5.3.1 Incidencia técnica: Fallo en el hardware de desarrollo

Durante el mes de febrero, coincidiendo con el desarrollo del *core* del sistema, uno de los miembros del equipo sufrió una interrupción crítica debido a un fallo técnico en su estación de trabajo (problemas de almacenamiento que derivaron en la inoperatividad del sistema operativo). Esta incidencia supuso la pérdida de una semana de capacidad de desarrollo para dicho integrante.

Para mitigar el impacto en el cronograma global y evitar un retraso en la entrega del *Sprint*, se aplicaron las siguientes medidas de contingencia:

- **Redistribución dinámica de la carga:** El segundo integrante asumió las tareas críticas que se encontraban en la ruta principal de esa iteración, asegurando que el flujo de trabajo del servidor no se detuviera.
- **Sincronización y continuidad:** Una vez restaurado el entorno de trabajo, el miembro afectado pudo retomar su actividad de forma inmediata sobre la base de código ya avanzada por su compañero. La estructura horizontal del equipo y el conocimiento *Full Stack* compartido permitieron que esta transición fuera transparente y sin pérdida de contexto técnico.

Como resultado, la desviación temporal fue absorbida por el propio equipo sin afectar a la fecha de finalización del hito mensual, validando la eficacia de la estructura organizacional elegida.

5.3.2 Conciliación académica y personal

Otra desviación potencial fue la gestión del tiempo durante los picos de carga de trabajo académico. La medida correctiva aplicada fue la intensificación de las reuniones de sincronización (*Daily Stand-ups*) para ajustar los objetivos del *Sprint* de forma realista semana a semana. El compromiso con un flujo de trabajo constante ha permitido que, a

pesar de las situaciones personales o de estudio, el proyecto no haya sufrido periodos de inactividad, manteniendo una entrega de valor continua desde el inicio del semestre.

1
2

COSTES

1

2 *The good parts of a book may be only something a writer is lucky enough to overhear or it may be the wreck*
3 *of his whole damn life – and one is as good as the other.*

4

Ernest Hemingway (1899–1961),

5

Novelist

6

7

8

R esumen de lo que va a ocurrir en el capítulo. ¿Cuál es el objetivo que tenemos con este capítulo?

6.1 RESUMEN DE COSTES DEL PROYECTO

1

Resumen del proyecto	
Costes de personal	5.045 €
Sueldo neto	2.030 €
Impuestos	1.000 €
Costes sociales	2.015 €
Costes materiales	560 €
Costes indirectos	450 €
TOTAL	8.000 €

Cuadro 6.1: Tabla resumen de costes

6.2 COSTES DE PERSONAL

2

Ya hablaremos de esto

3

6.3 COSTES MATERIALES

4

Y de esto también. Ver Sección §7.3.

5

6.4 COSTES INDIRECTOS

6

Y esto es una fiesta

7

PARTE III

DESARROLLO DEL PROYECTO

ANÁLISIS Y DISEÑO DEL SISTEMA

The good parts of a book may be only something a writer is lucky enough to overhear or it may be the wreck of his whole damn life – and one is as good as the other.

*Ernest Hemingway (1899–1961),
Novelist*

En este capítulo se definen el alcance funcional y técnico del sistema. En primer lugar, se establece el catálogo de requisitos funcionales y no funcionales, delimitando las fronteras del sistema. A continuación, se modela la interacción de los usuarios mediante diagramas de Casos de Uso y de Secuencia. Finalmente, se describe el diseño de la arquitectura de software y el modelo de datos diseñado para garantizar el rendimiento en tiempo real.

7.1 ANÁLISIS DE REQUISITOS 1

La fase de análisis tiene como objetivo definir el contrato del sistema, estableciendo qué debe hacer la aplicación (requisitos funcionales) y bajo qué restricciones de calidad, rendimiento y seguridad debe operar (requisitos no funcionales). 2 3 4

7.1.1 Requisitos Funcionales y Actores 5

Para la definición del Producto Mínimo Viable (MVP) de *King and Peasant*, se han identificado dos perfiles de interacción o **Actores** principales: 6 7

- **Usuario No Autenticado (Invitado):** Entidad que interactúa con las capas públicas del sistema, con el único objetivo de registrarse o autenticarse. 8 9
- **Jugador (Usuario Registrado):** Actor principal que posee una sesión activa y tiene acceso a la totalidad de las mecánicas del juego y módulos sociales. 10 11

Las características esenciales del sistema se estructuran en los siguientes módulos funcionales: 12 13

1. **Gestión de Identidad:** Registro, inicio de sesión seguro y gestión del perfil del usuario. 14 15
2. **Ecosistema Social:** Sistema de peticiones de amistad y chat en tiempo real dentro de una partida. 16 17
3. **Infraestructura de Lobbies:** Creación de salas de juego (públicas y privadas). 18
4. **Motor de Juego (Core):** Lógica asimétrica de cartas, gestión de turnos y sincronización bidireccional del estado del tablero. 19 20

7.1.2 Requisitos No Funcionales (RNF) 21

Dada la naturaleza interactiva del proyecto, los requisitos no funcionales resultan críticos para la viabilidad del sistema. Se han definido las siguientes restricciones técnicas: 22 23

- **Arquitectura y Stack:** El cliente debe construirse como una *Single Page Application* (SPA) con *React 18* y *TypeScript*. El servidor utilizará *Node.js* (*Express*) bajo una arquitectura modular. El código fuente se organizará estrictamente como un 24 25 26

Monorepositorio mediante *npm workspaces* para compartir dependencias entre cliente y servidor.

- **Rendimiento:** El sistema exige **baja latencia**. Las acciones de la partida deben procesarse en tiempo real utilizando el protocolo *WebSockets* (*Socket.io*) y apoyándose en *Redis* para el estado volátil. Asimismo, todo el entorno debe estar contenedorizado con *Docker* para garantizar un despliegue y escalabilidad ágiles.
- **Seguridad:** La autenticación se basará en *JSON Web Tokens* (JWT). Las credenciales se protegerán mediante algoritmos de *hashing* unidireccional (*bcryptjs*) y se aplicarán políticas CORS estrictas. Los secretos del sistema se aislarán mediante variables de entorno (`.env`).
- **Fiabilidad y QA:** Se requiere un flujo de Integración y Despliegue Continuo (CI/CD) automatizado mediante *GitHub Actions*. La calidad del código se garantizará con pruebas unitarias en el *backend* (*Jest*) y en el *frontend* (*Vitest*), realizando despliegues automáticos en entornos de *Staging* y Producción.

7.1.3 Reglas de Negocio y Restricciones de Dominio

Para comprender el contexto funcional completo de los requisitos previamente descritos, se recomienda consultar el manual íntegro del juego en el Anexo correspondiente[cite: 5, 6]. *King and Peasant* es un juego asimétrico [cite: 7] cuyas mecánicas imponen un conjunto de reglas de negocio estrictas. A continuación, se detalla cómo estas reglas condicionan la arquitectura y el comportamiento del servidor:

- **RN-01 (Estructura de la Partida en Eras):** La resolución del juego no se basa en una única ronda, sino que está dividida en “Eras”, proclamándose vencedor global el jugador que consiga ganar 2 Eras[cite: 10, 11, 58, 60]. *Restricción Arquitectónica:* El sistema debe gestionar un metacontrolador de estado que contabilice las victorias acumuladas y orqueste el reseteo completo del tablero, mazos y manos (Setup) al finalizar una Era, sin necesidad de destruir y recrear la sala de juego (*Lobby*)[cite: 59].
- **RN-02 (Capacidad de Tablero Asimétrica):** Las reglas dictan que el Rey tiene una restricción física en la zona de juego (*Town*) de un máximo de 3 cartas de Guardia [cite: 64, 76], mientras que el Campesino puede desplegar un número ilimitado de Rebeldes[cite: 76]. *Restricción Arquitectónica:* Los validadores de entrada del

servidor deben aplicar lógicas asimétricas dependiendo de la facción del emisor antes de procesar un evento de despliegue en la base de datos en memoria.	1 2
■ RN-03 (Restricción Espacial de Infiltración): El Campesino posee mecánicas críticas de infiltración en el mazo con cartas como el Señuelo o el Asesino. El reglamento exige que la carta objetivo se encuentre obligatoriamente desplegada boca abajo en el <i>Town</i> para poder efectuar esta acción[cite: 73, 74]. <i>Restricción Arquitectónica:</i> El motor de reglas (<i>Game Service</i>) debe rechazar cualquier petición de infiltración que provenga de la mano del jugador o si la carta ya ha sido revelada públicamente[cite: 74], garantizando la seguridad del estado frente a peticiones maliciosas del cliente.	3 4 5 6 7 8 9 10
■ RN-04 (Cierre de Turno y Flujo Asimétrico): El flujo de turno es radicalmente distinto entre actores: el Rey finaliza obligatoriamente su turno robando una carta [cite: 70], mientras que para el Campesino robar es una acción voluntaria más de su repertorio[cite: 48, 92]. <i>Restricción Arquitectónica:</i> El controlador de estado debe ser capaz de inyectar acciones automáticas en la pila de resolución del Rey, a la vez que mantiene un sistema de interrupción manual (espera de <i>payload</i>) para el Campesino.	11 12 13 14 15 16 17
■ RN-05 (Atomicidad de las Condiciones de Victoria): Las condiciones para ganar son asimétricas; por ejemplo, el Rey vence si se vacía el mazo o el Asesino es descartado [cite: 50, 51, 52], mientras que el Campesino gana si el Rey roba al Asesino[cite: 53, 54]. <i>Restricción Arquitectónica:</i> El servidor está obligado a invocar un servicio de auditoría tras cada modificación individual del estado (<i>listener</i> atómico), interrumpiendo cualquier evento encolado si se dispara el <i>trigger</i> de victoria de una facción.	18 19 20 21 22 23 24

7.2 MODELADO DEL SISTEMA 25

Para representar gráficamente las interacciones descritas en los requisitos, se ha utilizado el Lenguaje Unificado de Modelado (UML), estandarizando la comprensión de la arquitectura. 26
27
28

7.2.1 Diagrama de Casos de Uso 29

En la Figura §7.1 se detalla el comportamiento del sistema desde el punto de vista del usuario. Se observa cómo el *Jugador* tiene la capacidad de acceder al ecosistema central 30
31

1 dado que tiene una sesión activa. Destaca la modularidad del caso de uso principal
 2 (“Jugar Partida”), el cual incluye («include») obligatoriamente la mecánica de jugar
 3 cartas, y se extiende («extend») de forma opcional con acciones como el chat en vivo o la
 4 rendición voluntaria.

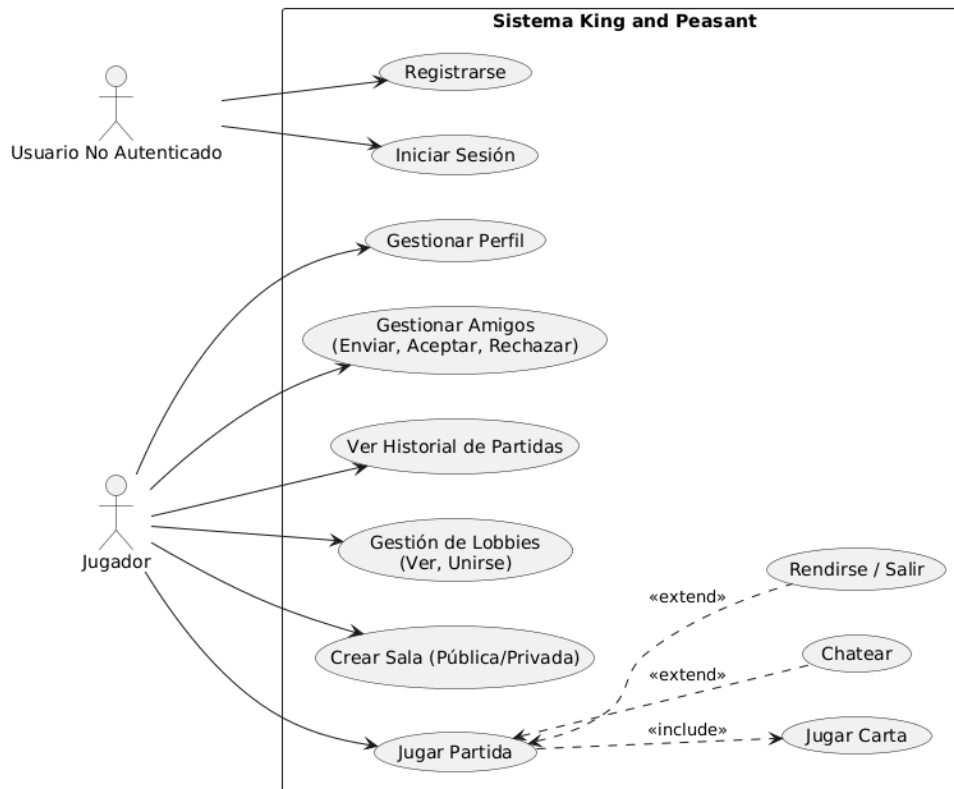


Figura 7.1: Diagrama General de Casos de Uso del sistema.

5 7.2.2 Comportamiento Dinámico: Flujo de Ejecución

6 Dado que los casos de uso estáticos no muestran el orden cronológico de las opera-
 7 ciones, se ha elaborado un Diagrama de Secuencia (Figura §7.2) que ilustra la travesía
 8 completa de un usuario: desde que no tiene cuenta hasta que juega una carta.

9 El diagrama evidencia la transición entre el protocolo HTTP y el protocolo Web-
 10 Sockets. Mientras que el registro y la creación de la sala (*Lobby*) se resuelven mediante
 11 peticiones REST tradicionales que persisten en *MariaDB*, el inicio de la partida desencade-
 12 na una actualización en la base de datos en memoria (*Redis*). A partir de ese punto,
 13 acciones iterativas como enviar un mensaje o mover una carta se transmiten mediante
 14 eventos asíncronos (*Broadcast*), logrando la latencia nula exigida en los requisitos no
 15 funcionales.

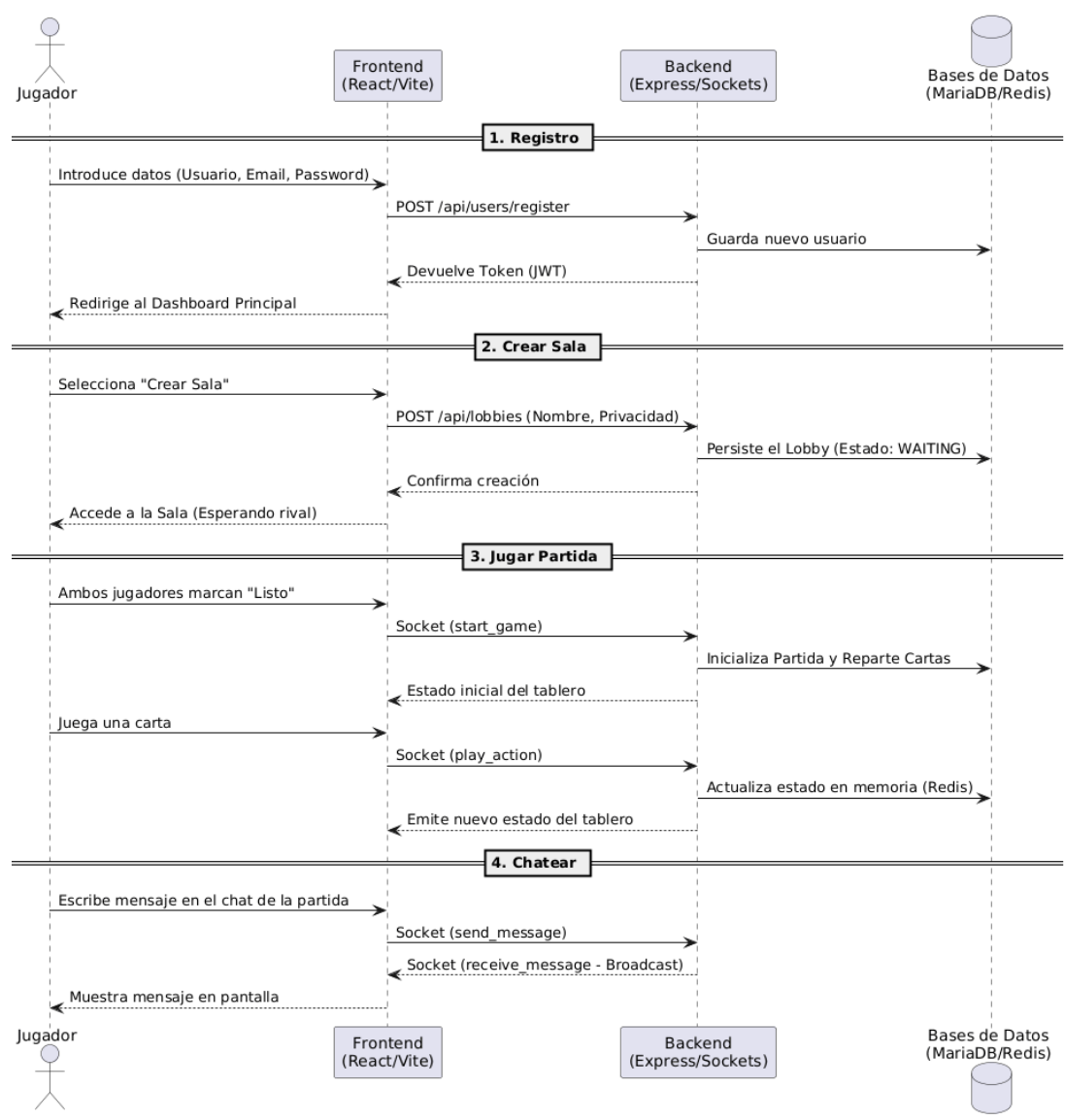


Figura 7.2: Diagrama de Secuencia: Registro, creación de sala y flujo de partida.

7.3 DISEÑO DE LA ARQUITECTURA Y EL SISTEMA

El diseño de la arquitectura del sistema constituye el nucleo estructural del proyecto. En este apartado se describe la estructura global adoptada, los patrones de diseño aplicados en cada subsistema y la justificación técnica de las decisiones tomadas frente a otras alternativas arquitectónicas disponibles.

En la Figura §7.3 se ilustra el esquema general del sistema, basado en un modelo cliente-servidor fuertemente desacoplado pero integrado bajo un mismo flujo de

1 desarrollo.

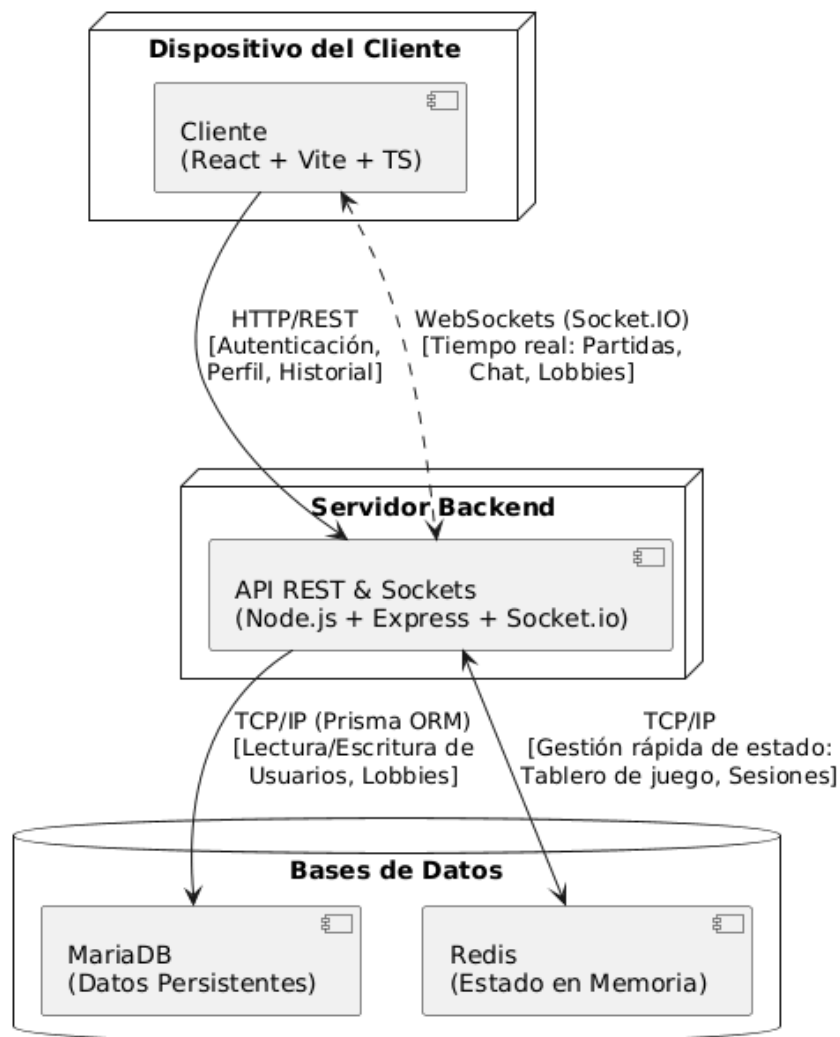


Figura 7.3: Diagrama de Arquitectura global del sistema y flujos de comunicación.

2 7.3.1 Estrategia Organizativa: Monorepositorio

3 En lugar de segregar el cliente y el servidor en repositorios de control de versiones
4 independientes (polirepositorio), se tomó la decisión arquitectónica de utilizar un
5 **Monorepositorio** gestionado de forma nativa mediante *npm workspaces*.

6 Esta alternativa se eligió principalmente por su capacidad para unificar y simplificar
7 la gestión del ciclo de vida del desarrollo. Al agrupar los módulos de *client* y *server* en un
8 único entorno, se facilita la configuración de la infraestructura local (bases de datos en
9 Docker) y se centraliza la orquestación de comandos. Esto permite que procesos como

la instalación de dependencias, la ejecución de pruebas (*testing*) y el despliegue local de toda la pila tecnológica (Frontend, Backend, MariaDB y Redis) se realicen con un solo comando desde el directorio raíz, reduciendo la fricción en el entorno de desarrollo y en los flujos de Integración Continua (CI/CD).

7.3.2 Arquitectura del Servidor (Backend)

El servidor se ha diseñado huyendo del patrón de monolito tradicional, apostando por una **Arquitectura por Capas** (*Layered Architecture*) estricta. Esta decisión fomenta el principio de responsabilidad única y facilita la creación de pruebas unitarias al aislar el enrutamiento de la lógica funcional. El flujo se divide en:

- 1. **Capa de Enrutamiento:** Define los *endpoints* HTTP.
- 2. **Capa de Controladores:** Procesa y sanea las peticiones.
- 3. **Capa de Servicios:** Encapsula toda la lógica de negocio compleja de forma agnóstica a la red.

Paradigma de Comunicación Bidireccional: Para el ecosistema de juego, se descartaron alternativas arquitectónicas como *Short Polling* o *Server-Sent Events* (SSE). El *Polling* tradicional generaría una sobrecarga HTTP inasumible y latencia en un juego por turnos. Por ello, se justificó el uso exclusivo de **WebSockets** (*Socket.io*), estableciendo un canal persistente, bidireccional y de muy baja latencia, crítico para sincronizar eventos como el robo de cartas o las notificaciones del chat en milisegundos.

7.3.3 Arquitectura del Cliente (Frontend)

El cliente se ha modelado como una *Single Page Application* (SPA) desarrollada en *React*. En lugar de acoplar la lógica de negocio a la capa de vista, se ha optado por una arquitectura de componentes funcionales apoyada en **Custom Hooks**.

Para la propagación del estado de sesión y de la partida, se descartó el envío manual de propiedades en cascada (*prop drilling*), adoptando el patrón **Context API**. Esto permite mantener un árbol de componentes limpio y optimizado, donde cada nodo accede únicamente a los datos que necesita, mejorando la mantenibilidad a largo plazo.

7.3.4 Arquitectura de Persistencia Híbrida

Para la persistencia de la información del sistema se ha diseñado un modelo de datos relacional robusto, gestionado a través de *Prisma ORM*. Este modelo soporta la lógica de negocio subyacente, garantizando la integridad referencial y facilitando las consultas complejas.

El esquema de la base de datos (*MariaDB*) gravita en torno a cinco entidades principales y sus relaciones, tal y como se ilustra en el diagrama Entidad-Relación (Figura §7.4).

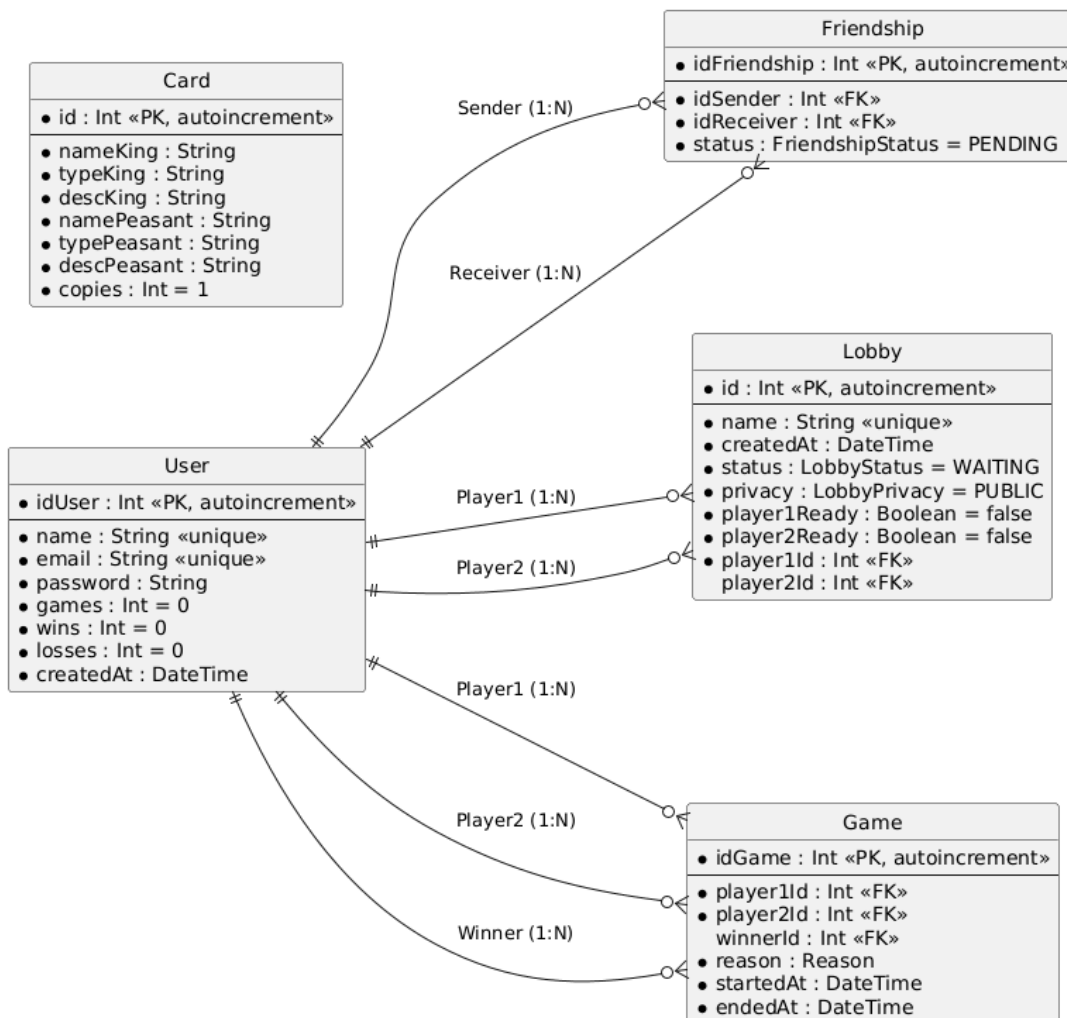


Figura 7.4: Diagrama Entidad-Relación del sistema.

Entidades Principales	1
■ Usuario (<i>User</i>): Es la entidad central del ecosistema. Además de almacenar las credenciales de acceso (correo y contraseña protegida), centraliza las estadísticas globales del jugador en la plataforma, manteniendo un conteo actualizado de partidas jugadas, victorias y derrotas. Actúa como nodo relacional para el resto de entidades del sistema.	2 3 4 5 6
■ Amistad (<i>Friendship</i>): Modela la red social del juego. En lugar de utilizar una relación binaria simple de existencia, se ha diseñado con un patrón de estado mediante el enumerado <code>FriendshipStatus</code> (<i>ACCEPTED</i> , <i>PENDING</i> , <i>DENIED</i>). Esto permite mantener un control transaccional sobre quién envió la petición (<i>Sender</i>) y quién la recibió (<i>Receiver</i>), habilitando la gestión asíncrona de invitaciones.	7 8 9 10 11
■ Sala (<i>Lobby</i>): Aunque la concurrencia y el estado en tiempo real de las salas se delega en la base de datos en memoria (<i>Redis</i>) por motivos de latencia, esta entidad define la estructura persistente de la sala. Almacena metadatos críticos como el nivel de privacidad (<i>PUBLIC/PRIVATE</i>), el estado actual (<i>WAITING/ONGOING</i>) y la confirmación de preparación (<i>Ready</i>) de los dos jugadores vinculados.	12 13 14 15 16
■ Carta (<i>Card</i>): Define los elementos jugables del sistema. Resulta destacable el enfoque de diseño asimétrico aplicado a esta entidad. Para optimizar el esquema y evitar la duplicidad de tablas entre las dos facciones del juego (Rey y Campesino), se ha optado por un modelo unificado. Cada registro contiene simultáneamente los atributos (nombre, tipo y descripción) de su variante como Rey y su variante como Campesino, centralizando la configuración de los mazos.	17 18 19 20 21 22
■ Partida (<i>Game</i>): Constituye el registro histórico del sistema. Relaciona a dos usuarios como oponentes y registra de forma inmutable el identificador del ganador, así como las marcas de tiempo de inicio y fin.	23 24 25
Estado en Memoria (<i>Redis</i>) y Estructura del <i>GameState</i>	26
Mientras que <i>MariaDB</i> gestiona los datos inmutables y a largo plazo, el núcleo de la partida en vivo reside en <i>Redis</i> . El estado del juego (<i>gameState</i>) es un objeto JSON complejo que actúa como la única fuente de verdad (<i>Single Source of Truth</i>) durante el transcurso de una partida.	27 28 29 30
Este objeto almacena propiedades persistentes durante el encuentro (como el turno actual, la era, las puntuaciones y los mazos) y atributos volátiles u opcionales que modelan la lógica interna del servidor:	31 32 33

- `pendingAction`: Objeto temporal que pausa el flujo principal de la partida cuando el sistema requiere una decisión asíncrona de un jugador (por ejemplo, seleccionar un objetivo tras jugar una carta de efecto).
- Pila de resolución (`queuedKingMobilize` y `queuedPeasantDispatch`): Estructuras que permiten encadenar y resolver múltiples acciones secuenciales de forma ordenada.
- `seenByKing`: Bandera booleana (*flag*) introducida por mecánicas específicas de espionaje, que permite asimetría en la visibilidad de una misma carta.
- `lastEvent`: Etiqueta temporal utilizada por el motor interno para procesar condiciones de victoria o cambios de fase, la cual se purga antes de emitir cualquier estado.

Seguridad y Patrón de Transferencia de Datos (DTO)

Transmitir el *gameState* crudo a los clientes supondría una vulnerabilidad crítica, ya que un jugador podría inspeccionar el tráfico de *WebSockets* mediante las herramientas de desarrollador del navegador y descubrir las cartas ocultas de su oponente.

Para mitigar este riesgo desde el diseño de la arquitectura, se ha implementado el patrón **Data Transfer Object (DTO)**. El backend nunca expone el estado original de *Redis*. En su lugar, el servicio de juego clona el estado y ejecuta una función de transformación que genera dos vistas asimétricas: `dtoKing` y `dtoPeasant` (Figura 7.5).

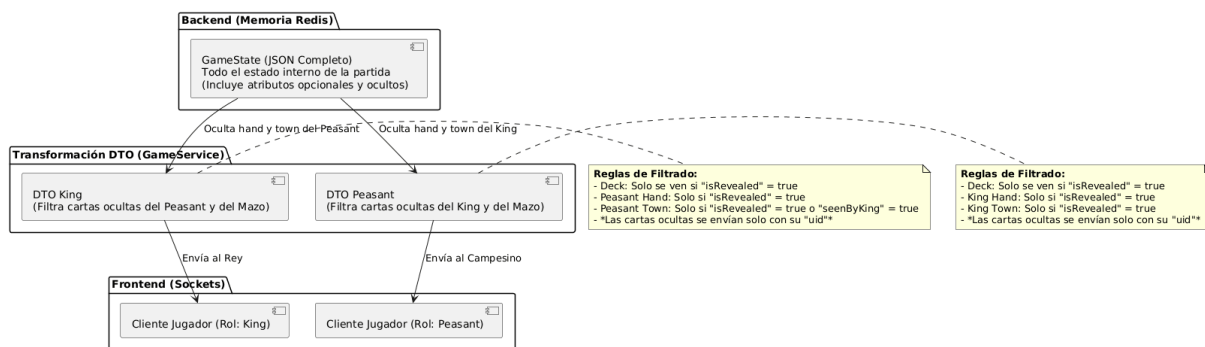


Figura 7.5: Arquitectura de seguridad y filtrado de estado mediante el patrón DTO.

Las reglas de filtrado aplicadas por el servidor son estrictas:

1. **Mazo Global (*Deck*):** Cualquier carta no revelada (`isRevealed: false`) es cen- 1
surada, transmitiendo al cliente únicamente su identificador único (`uid`). 2
 2. **Censura Asimétrica:** El DTO del Rey recibe la mano y la ciudad del Campesino 3
censurados, a menos que la carta haya sido revelada públicamente o interceptada 4
(`seenByKing: true`). De forma recíproca, el DTO del Campesino recibe los datos 5
del Rey completamente censurados si no cumplen las condiciones de visibilidad. 6
- Esta capa de transformación garantiza la integridad competitiva del sistema, ase- 7
gurando que el cliente de React solo procese y renderice la información a la que tiene 8
acceso legítimo. 9

IMPLEMENTACIÓN Y DESARROLLO DEL SISTEMA

The good parts of a book may be only something a writer is lucky enough to overhear or it may be the wreck of his whole damn life – and one is as good as the other.

*Ernest Hemingway (1899–1961),
Novelist*

Este capítulo documenta el ciclo de vida del desarrollo de software del proyecto King and Peasant, narrando la evolución técnica desde la orquestación inicial de la infraestructura hasta la estabilización funcional del motor asimétrico. El objetivo principal es exponer las decisiones arquitectónicas, algorítmicas y de diseño adoptadas a lo largo de 16 iteraciones iterativas e incrementales. Se detalla la resolución de retos complejos de ingeniería, como la sincronización bidireccional mediante WebSockets, la protección de estados asimétricos a través de DTOs, la gestión de memoria en lógicas de cascada y la automatización del aseguramiento de la calidad. A través de este análisis progresivo, se justifica de forma práctica cómo los requerimientos de fiabilidad, rendimiento y mantenibilidad han sido satisfechos en el producto final.

8.1 CONFIGURACIÓN DEL ENTORNO Y ARQUITECTURA BASE 1

Flujo de Interacción y Arquitectura 2

Esta fase establece la orquestación inicial del monorepositorio utilizando espacios de trabajo (*npm workspaces*) para unificar el desarrollo del cliente (*Vite/React*) y el servidor (*Node.js/Express*). La arquitectura se basa en contenedores *Docker*, permitiendo un despliegue aislado de los servicios de infraestructura (*MariaDB* y *Redis*) para garantizar la paridad de entornos locales y facilitar la futura integración continua. 3 4 5 6 7

Estructura de Directorios 8

```
KingAndPeasant/ 10
|-- docker-compose.yml 11
|-- packages/ 12
|   |-- client/ 13
|   |   |-- src/ 14
|   |   +-- package.json 15
|   +-- server/ 16
|       |-- src/ 17
|       +-- package.json 18
+-- package.json 19
```

Snippets Estratégicos de Código 20

```
# docker-compose.yml 22
version: '3.8' 23
services: 24
  mariadb: 25
    image: mariadb:10.11 26
    # ... configuracion interna omitida para resumir el contenido 27
  redis: 28
    image: redis:7.0 29
    # ... configuracion interna omitida 30
```

1 8.1.1 Iteración 1: Configuración de Entornos y Metodología Base

2 **Objetivos de la Iteración**

3 El objetivo primordial de esta primera iteración consistió en establecer los cimientos
4 arquitectónicos e infraestructurales del proyecto *King and Peasant*. Se buscó preparar
5 un entorno de desarrollo robusto que garantizara la paridad entre los entornos de los
6 distintos desarrolladores y sentar las bases de la metodología ágil que guiará el resto
7 del ciclo de vida del software.

8 **Desarrollo e Implementación**

9 Durante este periodo, las tareas realizadas se centraron en la configuración inicial del
10 entorno para la orquestación técnica del sistema. A nivel de equipo, se inicializó el
11 repositorio remoto bajo una estructura de monorepositorio. Adicionalmente, se redactó
12 el documento de metodología en Markdown para estandarizar la creación de ramas, los
13 *commits* y el seguimiento de *issues*, en cumplimiento con los estándares organizativos
14 del proyecto.

15 A nivel técnico, satisfaciendo el requisito de arquitectura (**RNF-ARQ**), se implementó
16 la orquestación del sistema mediante *Docker* y *Docker Compose*. La creación de los archivos
17 `Dockerfile` permitió el despliegue aislado de los entornos de ejecución base, asegurando
18 una configuración sin conflictos de dependencias para el posterior desarrollo de la lógica
19 de negocio (véase la Figura §8.1).

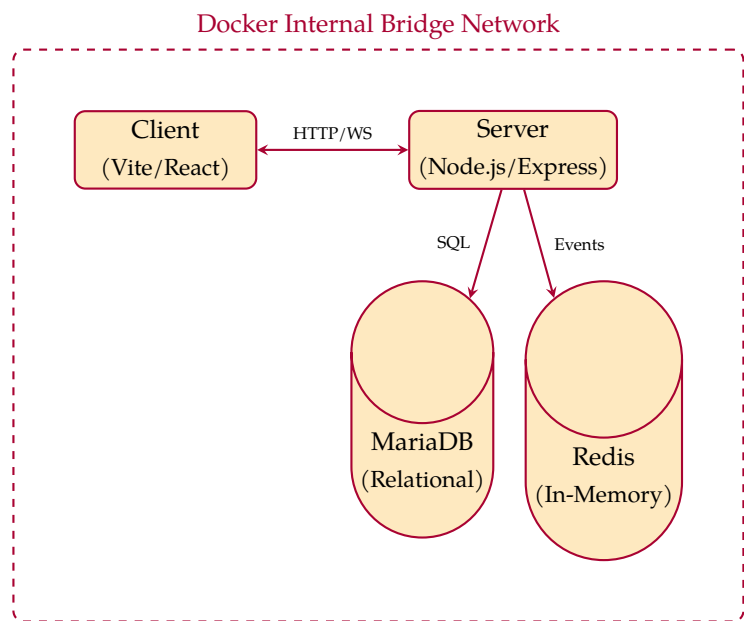


Figura 8.1: Diagrama de topología de contenedores de la infraestructura base, ilustrando la red interna (bridge) entre el servidor, MariaDB y Redis.

Profundización Técnica: Orquestación Local y Workspaces de NPM	1
Para asegurar la cohesión del monorepositorio, se estructuró el proyecto utilizando	2
<i>npm workspaces</i> , agrupando el cliente (<i>Vite/React</i>) y el servidor (<i>Node.js</i>) bajo un único	3
árbol de dependencias. La orquestación se materializó a través de <i>scripts</i> centralizados	4
en el <i>package.json</i> raíz, combinando <i>concurrently</i> para levantar múltiples entornos de	5
desarrollo en paralelo y <i>Docker Compose</i> para inicializar la capa de persistencia. Esto	6
permitió que, mediante un único comando (<code>npm run start:local</code>), el sistema validara	7
la salud de los contenedores y ejecutara las migraciones de base de datos antes de	8
exponer los puertos, eliminando la fricción de configuración inicial.	9
Desviaciones y Retos Técnicos	10
Durante la orquestación inicial surgieron conflictos de red que impedían el arranque	11
limpio de los servicios en los contenedores. El problema se solucionó unificando la red	12
interna bajo un driver <i>bridge</i> personalizado para visibilidad entre contenedores.	13
Estado de Cierre	14
La iteración concluyó satisfactoriamente tras alcanzar todos los hitos previstos. La	15
infraestructura base y las políticas del repositorio quedaron operativas, eliminando	16
cualquier fricción técnica inicial y permitiendo que el equipo comenzara el desarrollo	17
de la capa de datos en los <i>sprints</i> posteriores.	18

8.1.2 Iteración 2: Diseño de Interfaces y Orquestación de Datos

Objetivos de la Iteración

El objetivo establecido para este ciclo, centrado en las responsabilidades individuales de infraestructura y diseño visual, fue sentar las bases de la identidad del proyecto *King and Peasant* mediante la elaboración de *mock-ups* detallados y la orquestación de las bases de datos en el entorno local.

Desarrollo e Implementación

El trabajo de este ciclo arrancó con la elaboración de los *mock-ups* completos de las pantallas clave del ecosistema, incluyendo los flujos de autenticación (Login/Registro), las salas de espera (*Lobbies*), el perfil del usuario, el menu principal y el tablero de la partida asimétrica. Esta labor proporcionó una visión estructural clara para el desarrollo en cliente, como se ilustra en la Figura §8.2.

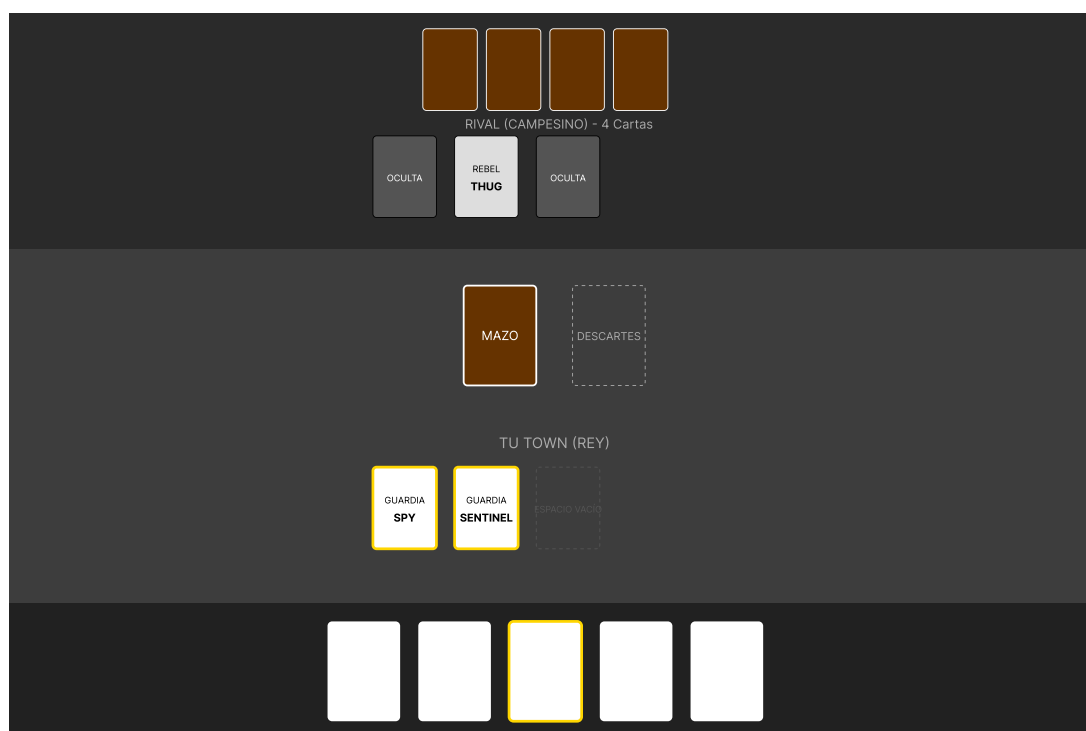


Figura 8.2: Captura de los mock-ups de la interfaz principal: Tablero de juego asimétrico y salas de espera (*Lobbies*).

A nivel de infraestructura, y en cumplimiento del requisito de rendimiento (**RNF-REN**), se amplió la configuración de *Docker Compose* para incluir el despliegue aislado

de los servicios de *MariaDB* (base de datos relacional encargada de la lógica de negocio) y *Redis* (almacenamiento en memoria para eventos en tiempo real).

Profundización Técnica: Configuración Aislada de Bases de Datos con Docker

La integración de múltiples motores de bases de datos exigió un diseño cauteloso en el archivo `docker-compose.yml`. Se configuraron volúmenes persistentes (*Volumes*) tanto para *MariaDB* (`/var/lib/mysql`) como para *Redis* (`/data`), asegurando que la reinicialización de los contenedores no provocara la pérdida de información crítica en desarrollo. Adicionalmente, se implementaron comprobaciones de estado (*healthchecks*) con comandos nativos como `mysqladmin ping` y `redis-cli ping`, lo que garantiza que los servicios de *backend* dependientes (*depends_on*) no intenten establecer conexiones hasta que los motores de persistencia estén completamente operativos y listos para recibir transacciones.

Desviaciones y Retos Técnicos

Al integrar el almacenamiento en memoria surgieron bloqueos temporales en la comunicación de los servicios locales. La solución fue corregir el mapeo de puertos exponiendo el 6379 hacia el *localhost* en el `docker-compose.yml`, garantizando así la paridad de entornos.

Estado de Cierre

Tras resolver los conflictos de red entre contenedores, las tareas de orquestación de bases de datos y diseño visual se completaron satisfactoriamente y se integraron en la rama principal, dejando la plataforma preparada para iniciar la estructuración del ORM.

8.2 AUTENTICACIÓN Y MODELADO DE DATOS

Flujo de Interacción y Arquitectura

El flujo de autenticación implementa el patrón de diseño *Context API* en el *frontend* para inyectar globalmente el estado de sesión del usuario. A nivel de *backend*, se utiliza *Prisma ORM* como intermediario transaccional con la base de datos relacional. Las peticiones a rutas protegidas se validan mediante *middlewares* que decodifican *JSON Web Tokens* (*JWT*), logrando una arquitectura de seguridad robusta y desacoplada.

Estructura de Directorios

```
packages/  
|-- client/src/
```

```

1 |   +-- context/
2 |       +-- AuthContext.ts
3 +-- server/
4     |-- prisma/
5     |   +-- schema.prisma
6     +-- src/controllers/
7         +-- UserController.js

```

Snippets Estratégicos de Código

```

10 // packages/client/src/context/AuthContext.ts
11 interface AuthContextType {
12   user: User | null;
13   isLogin: boolean;
14   login: (userData: User) => void;
15   // ... logica interna omitida para resumir el contenido
16   socket: Socket | null;
17 }
18 export const AuthContext = createContext<AuthContextType | undefined>(undefined);
19
19 // packages/server/prisma/schema.prisma
20 model User {
21   idUser    Int        @id @default(autoincrement())
22   name      String     @unique
23   email     String     @unique
24   password  String
25   // ... resto de campos relacionales omitidos
26 }

```

8.2.1 Iteración 3: Modelado Estructural y Flujo de Negocio

Objetivos de la Iteración

El objetivo principal asignado para esta iteración en el proyecto *King and Peasant* se centró en la conceptualización teórica del dominio de datos, así como en el modelado formal del flujo que gobernaría la asimetría de las partidas.

Desarrollo e Implementación

La implementación de esta etapa se articuló en torno al diseño de la arquitectura de la

información. Se elaboró el modelo Entidad-Relación y el diagrama UML correspondiente a la base de datos relacional. De forma paralela, se modelaron los estados de la partida, los turnos intercalados y las mecánicas asimétricas mediante diagramas de flujo.

Profundización Técnica: Diseño del Esquema Relacional de Entidades

El diseño estructural requirió abstraer la asimetría del juego a un modelo de datos cohesivo. El esquema relacional definió entidades independientes para `User`, `Friendship`, `Lobby` y `Game`. La complejidad se centró en la entidad `Card`, la cual fue modelada para contener tanto los atributos del bando del Rey (`nameKing`, `typeKing`) como los del Campesino (`namePeasant`, `typePeasant`) en un mismo registro. Esto facilitó enormemente la futura gestión del mazo único compartido sin duplicar información estructural, aprovechando la naturaleza de cartas de "doble cara" del juego. Las relaciones múltiples reflexivas en la entidad `User` (como `gamesAsPlayer1` o `friendshipsSent`) sentaron las bases para consultas eficientes de historiales y marcadores.

Estado de Cierre

Los objetivos de modelado de datos y flujos lógicos se alcanzaron con éxito, sin registrarse desviaciones temporales ni bloqueos técnicos, dejando el diseño listo para ser implementado en la siguiente iteración.

8.2.2 Iteración 4: Implementación del Modelo Estructural para los usuarios

Objetivos de la Iteración

El objetivo principal para esta iteración se basó en la implementación técnica del modelo de datos en la base de datos relacional haciendo uso del modelo Entidad-Relación y el diagrama UML correspondiente.

Desarrollo e Implementación

En cumplimiento con el requisito de arquitectura (**RNF-ARQ**), este diseño conceptual se trasladó al entorno de desarrollo escribiendo los esquemas iniciales mediante *Prisma ORM*, ejecutando migraciones sobre el despliegue aislado de *MariaDB*. Además de asegurar el correcto funcionamiento de la base de datos al ejecutar el proyecto utilizando *docker*.

Profundización Técnica: Migraciones de Esquema mediante Prisma ORM

La transición del modelo conceptual a la infraestructura real se orquestó utilizando *Prisma*. Al definir el esquema (`schema.prisma`), se generó automáticamente un cliente tipado para *Node.js*, lo que proporciona autocompletado y seguridad de tipos en la capa de controladores. Las migraciones (`prisma migrate`) actuaron como un sistema de

control de versiones sobre el esquema relacional en *MariaDB*, garantizando que cualquier cambio en la estructura de tablas, índices (como `@@unique([idSender, idReceiver])` en las amistades) o relaciones de claves externas se aplicase de forma atómica y auditable en todos los entornos de los desarrolladores sin intervención manual.

Estado de Cierre

Los objetivos de implementación del modelo de datos se alcanzaron con éxito, dejando el modelo listo para ser utilizado en la siguiente iteración.

8.2.3 Iteración 5: Gestión de Identidad, Seguridad y Perfiles

Objetivos de la Iteración

El objetivo primordial de este ciclo consistió en implementar de forma integral la capa de gestión de usuarios del ecosistema. Esto abarcó tanto la seguridad del acceso (registro y autenticación de credenciales) como la administración de la identidad digital del jugador, proporcionando una zona privada para la edición de datos personales y la visualización de estadísticas de juego.

Desarrollo e Implementación

Durante esta fase se abordó la construcción de las vistas operativas de *Login*, *Register* y el panel de *Profile*. Dando cumplimiento al requisito de seguridad (**RNF-SEG**), se amplió la lógica de negocio del servidor para la gestión de *JSON Web Tokens* (JWT), asegurando el encriptado unidireccional de contraseñas y la firma de sesiones.

A nivel de persistencia, se desarrolló la lógica encargada de procesar y validar las actualizaciones de perfil en la base de datos relacional mediante *Prisma ORM*. En el cliente, se integraron formularios reactivos para la edición de datos y se maquetó la interfaz encargada de exponer las estadísticas históricas del usuario, consolidando así todo el flujo previo necesario para el inicio de las partidas.

Profundización Técnica: Autenticación Segura y Flujo JWT

El núcleo de la seguridad de cuentas se fundamentó en la emisión de *JSON Web Tokens*. Durante el registro, las contraseñas se ofuscan utilizando algoritmos de *hashing* con sal computacional (*bcrypt*) antes de ser persistidas. En la autenticación exitosa, el *backend* firma un JWT con una clave de servidor, empaquetando el identificador del usuario. El *frontend* inyecta este *token* como cabecera (*Bearer Authorization*) en cada petición HTTP protegida. Un *middleware* en *Node.js* intercepta estas peticiones, verificando la validez criptográfica de la firma y el ciclo de vida del *token* antes de permitir el acceso a controladores restringidos, como la edición del perfil.

Desviaciones y Retos Técnicos La integración de la seguridad y la gestión de perfiles presentó desafíos tanto en el cliente como en el servidor. En el *frontend*, surgieron dificultades con la validación de tokens y la persistencia del estado de sesión durante la navegación. Se resolvió implementando un Contexto Global de Autenticación (*AuthContext*) en *React*, gestionando el ciclo de vida del JWT mediante el patrón de Proveedor de Estado para proteger las rutas privadas de forma centralizada.

Paralelamente, en el *backend*, el diseño inicial de los controladores presentó conflictos lógicos al mezclar validaciones de entrada con lógica de persistencia. El obstáculo se superó mediante una refactorización orientada a la separación de responsabilidades, aislando la validación de esquemas de la lógica de actualización en *MariaDB* para estabilizar los servicios de cuenta.

Estado de Cierre La iteración concluyó satisfactoriamente con la capa de identidad plenamente operativa. Se lograron unificar los flujos de acceso y gestión de perfil, dejando la plataforma preparada para la introducción de las mecánicas sociales en tiempo real.

8.3 INTERACCIONES SOCIALES Y COMUNICACIÓN BIDIRECCIONAL

Flujo de Interacción y Arquitectura

Para resolver la latencia en la interacción social (sistema de amigos y futuras mecánicas de partida), se sustituye el modelo tradicional de *HTTP Polling* por una conexión persistente bidireccional basada en el protocolo *WebSockets* utilizando *Socket.io*. La arquitectura se modela mediante un *Socket Manager* en el servidor, implementado bajo un patrón *Singleton*, que gestiona la emisión de eventos (*Broadcast*) a salas específicas y propaga notificaciones en tiempo real a los clientes conectados.

Estructura de Directorios

```
packages/server/src/
|-- controllers/
|   +-- FriendshipController.js
+-- sockets/
    +-- LobbySocket.js
```

Snippets Estratégicos de Código

```

1 // packages/server/src/sockets/LobbySocket.js
2 const registerLobbyHandlers = (io, socket) => {
3   // ... configuracion interna omitida para resumir el contenido
4   socket.on("createLobby", async (lobbyName, isPrivate) => {
5     // ... logica de emision de eventos omitida
6   });
7 }

```

8.3.1 Iteración 6: Interacciones Sociales y Comunicación en Tiempo Real

Objetivos de la Iteración

El objetivo establecido para este ciclo en el proyecto *King and Peasant* fue implementar el componente social del ecosistema, diseñando un sistema de amistades entre usuarios e introduciendo la base técnica para la comunicación bidireccional necesaria en la futura partida.

Desarrollo e Implementación

El núcleo de ingeniería de esta iteración se destinó a desarrollar la lógica completa para gestionar las relaciones de usuarios. Se modelaron las entidades de amistad en la base de datos relacional y se desarrollaron los *endpoints* necesarios para enviar, aceptar y rechazar solicitudes. Para la visualización, se renderizó el *Dashboard* de amigos en el cliente. Paralelamente, y en cumplimiento del requisito de rendimiento (**RNF-REN**), se abordó una carga técnica considerable implementando *WebSockets* (*Socket.io*) en el servidor, sentando la base de la *lógica de negocio* asíncrona. Esta capa de comunicación permitió dotar al sistema de amistad de notificaciones en tiempo real, garantizando su fiabilidad (**RNF-FIA**) dentro del *despliegue aislado*. La justificación arquitectónica de esta decisión se detalla en la Tabla §8.1.

Profundización Técnica: Sincronización en Tiempo Real mediante WebSockets

La integración de una comunicación bidireccional exigió superar el paradigma tradicional de peticiones HTTP, requiriendo un canal persistente y de baja latencia. El principal desafío computacional consistió en mantener una única fuente de verdad (*Single Source of Truth*) sincronizada entre la persistencia en base de datos y la vista efímera del cliente. Para resolverlo, el algoritmo de emisión de eventos operó bajo las siguientes directrices:

- Se establece un mapa en memoria en el servidor que vincula el identificador del usuario (`userId`) con su `socket.id` de sesión.
- Tras completarse con éxito una transacción atómica en *MariaDB* (ej. aceptar una

- solicitud), el controlador notifica internamente al gestor de *Sockets*. 1
- El *Socket Manager* recupera el cliente activo y emite un evento tipado (*Broadcast* 2 dirigido) enviando el nuevo estado del *Dashboard*, forzando a la vista *React* a 3 re-renderizarse de forma reactiva sin necesidad de sondeo (*Polling*). 4

Análisis de valor: Comunicación por WebSockets	
Propuesta	Sustitución del paradigma HTTP tradicional (REST) por un canal persistente bidireccional basado en Socket.io.
Valor	Garantiza latencia casi nula para la sincronización de las mecánicas de partida y notificaciones de amistad.
Coste	Aumenta la complejidad de la infraestructura al requerir un <i>Socket Manager</i> y mayor consumo de memoria por conexiones abiertas.
Opciones	<i>HTTP Long Polling</i> o <i>Server-Sent Events (SSE)</i> , descartadas por sobrecarga de red y limitación unidireccional respectivamente.
Riesgos	Riesgo crítico de fugas de memoria (<i>memory leaks</i>) ante desconexiones no gestionadas de los clientes.
Deuda técnica	Obliga a implementar lógicas manuales de reconexión y ciclos de limpieza (<i>cleanup</i>) en el frontend (<i>React</i>).

Cuadro 8.1: Análisis de valor aportado sobre la arquitectura de comunicación mediante WebSockets.

- Desviaciones y Retos Técnicos** 5
- La introducción de los *WebSockets* generó retos técnicos en la capa de comunicación, 6 detectándose dificultades persistentes para establecer una conexión bidireccional estable. 7 La incidencia se resolvió gestionando el ciclo de vida de la conexión mediante retornos 8 de limpieza (*cleanup*) en los *hooks* de *React*, forzando la desconexión explícita para evitar 9 fugas de memoria o múltiples instancias por usuario. 10
- Estado de Cierre** 11
- A pesar de la complejidad que supuso orquestar los *Sockets* en el *frontend*, el sistema 12 de amistad quedó plenamente funcional con notificaciones asíncronas integradas en la 13 rama principal. El ciclo concluyó exitosamente, dejando el motor de comunicación listo 14 para iniciar el flujo principal de la partida. 15

1 8.4 ARQUITECTURA DEL MOTOR DE JUEGO ASIMÉTRICO

2 Flujo de Interacción y Arquitectura

3 El núcleo funcional del proyecto delega el estado asimétrico de las partidas a un servicio
 4 especializado (*Game Service*) que interactúa con la base de datos en memoria *Redis* para
 5 minimizar la latencia. El intercambio de información entre servidor y cliente se rige por
 6 Objetos de Transferencia de Datos (*DTO*), filtrando atributos confidenciales antes de
 7 emitir actualizaciones por *WebSockets*. Las mecánicas asíncronas, como *Pending Actions*,
 8 pausan y retoman el hilo de ejecución mediante identificadores únicos.

9 Estructura de Directorios

10

```
11 packages/
12 |-- client/src/pages/game/
13 |   |-- components/
14 |       +--- GameService.tsx
15 +--- server/src/
16     |-- services/
17     |   +--- GameService.js
18     +--- controllers/
19         +--- GameController.js
```

20 Snippets Estratégicos de Código

21

```
22 // packages/server/src/services/GameService.js
23 const playActionCard = async (gameId, targetData, playedCard, userRol, gam
24   // ... logica interna de validacion y estado en Redis omitida
25 }
26
27 const resolvePendingAction = async (gameId, userId, targetData) => {
28   // ... logica de resolucion transaccional omitida
29 }
```

30 8.4.1 Iteración 7: Documentación, Planificación y QA

31 Objetivos de la Iteración

32 El objetivo inicial de la iteración consistía en comenzar el desarrollo técnico del motor

de juego y el chat en partida. Sin embargo, el equipo decidió estratégicamente pivotar los esfuerzos hacia la gestión del proyecto para cumplir con los estándares académicos de *King and Peasant*.

Desarrollo e Implementación

Durante este periodo, las tareas técnicas se bloquearon temporalmente, dedicando el esfuerzo completo del equipo a la redacción y consolidación de la documentación de ingeniería del software. Se elaboró la Estructura de Desglose del Trabajo (EDT), la gestión de requisitos y el cronograma del proyecto. Asimismo, y dando cumplimiento al requisito de fiabilidad (RNF-FIA), se definió el Plan de Gestión de la Calidad (QA), estableciendo la estrategia integral de pruebas estáticas, unitarias y de integración que acompañarían a la *lógica de negocio* en el futuro.

Desviaciones y Retos Técnicos

Esta iteración representó en sí misma una desviación significativa respecto a la hoja de ruta técnica. La necesidad de documentar exhaustivamente el alcance funcional y arquitectónico del sistema supuso pausar la escritura de código para asegurar la viabilidad teórica del proyecto. Como resultado, las tareas originales relativas al modelo de datos de las cartas y a la comunicación bidireccional fueron trasladadas en su totalidad a los siguientes *sprints*.

Estado de Cierre

Los objetivos de documentación se completaron y validaron con éxito. El equipo generó todos los artefactos de gestión necesarios, estableciendo un marco de trabajo robusto y permitiendo retomar el desarrollo de la *lógica de negocio* en la siguiente semana.

8.4.2 Iteración 8: Refactorización y Estado Inicial de Partida

Objetivos de la Iteración

El objetivo de este *sprint* en *King and Peasant* fue realizar una reestructuración profunda de la base de código existente para mejorar su legibilidad y preparar la arquitectura para el complejo motor de juego, abordando también la configuración del estado inicial de la partida.

Desarrollo e Implementación

La prioridad técnica de este *sprint* consistió en la limpieza y reorganización del *frontend*. Dando cumplimiento al requisito de mantenibilidad (RNF-MAN), se unificó la nomenclatura de los componentes, se reestructuraron las carpetas de las vistas y se aislaron elementos reutilizables para reducir la deuda técnica. A nivel de orquestación de datos y

arquitectura (RNF-ARQ), se modelaron en la base de datos las entidades iniciales de las cartas (modelos y *seeders*) y se definieron los Objetos de Transferencia de Datos (DTO) necesarios para aislar la *lógica de negocio* y enviar un estado de juego seguro al cliente.

Profundización Técnica: Aislamiento de Información Mediante DTOs y Mappers

La asimetría del juego requiere que la información sea parcialmente ciega para los jugadores (el Campesino no debe conocer la mano del Rey y viceversa). Inicialmente, la serialización directa de los objetos del motor hacia JSON exponía datos privados por la red. Para resolver esta vulnerabilidad estructural, se introdujo el patrón de diseño *Data Transfer Object* (DTO). Se implementaron funciones adaptadoras (*mappers*) puras que reciben el estado absoluto del juego desde *Redis* y construyen un nuevo objeto purgado y específico para el *socket* receptor. De este modo, propiedades sensibles como la mano del rival se envían únicamente como conteos de longitud (números enteros), garantizando que el cliente no pueda inspeccionar el tráfico de red para obtener ventajas ilícitas.

Desviaciones y Retos Técnicos

El proceso de refactorización reveló brechas en el filtrado de datos, las cuales exponían información que debía permanecer oculta debido a la asimetría del juego (ej. *Rival town dto*). La solución fue implementar una función adaptadora (*mapper*) estricta antes de la serialización JSON para purgar las propiedades privadas del rival.

Estado de Cierre

La iteración concluyó con una base de código notablemente más robusta y organizada. Las refactorizaciones y la configuración del estado inicial de juego se fusionaron en la rama principal, mitigando la deuda técnica acumulada y dejando el terreno arquitectónico preparado para la implementación funcional del sistema de cartas.

8.4.3 Iteración 9: Desarrollo del Motor de Juego y Cartas de Acción

Objetivos de la Iteración

El objetivo establecido para este ciclo consistió en seguir completando el motor del juego de *King and Peasant*, centrándose específicamente en la implementación funcional de las cartas de acción y el complejo sistema de acciones pendientes (*Pending Actions*) para manejar la interactividad asimétrica de la partida.

Desarrollo e Implementación

En esta fase, el trabajo se dedicó a dotar de *lógica de negocio* al núcleo del juego. En el servidor, se construyeron los servicios (*Game Service*) y controladores (*Game Controller*) encargados de procesar, validar y resolver el uso de las cartas de ambas facciones,

validando continuamente la capacidad asimétrica del tablero (**RN-02**) y procesando de forma automática el cierre de turno según el bando (**RN-04**). Dando cumplimiento al requisito de arquitectura (**RNF-ARQ**), esta lógica se integró mediante la orquestación del estado en memoria y la persistencia relacional, manteniendo la *paridad de entornos* entre el desarrollo local y el *despliegue aislado*. En el cliente, se desarrollaron las interfaces para interactuar con la mano del jugador, los modales dinámicos para explorar la pila de descartes y el diccionario visual de acciones pendientes.

Profundización Técnica: Resolución de Acciones Pendientes (Pending Actions)

La asimetría del juego requiere mecánicas que pausan el flujo natural del turno, obligando a un jugador a tomar decisiones interactivas antes de que la partida pueda continuar. El problema computacional residía en congelar el estado del motor de forma segura en un entorno *Stateless* como HTTP, delegando el contexto temporal a *Redis*. El sistema de cola de acciones (*Pending Actions*) se diseñó así:

- Al detectarse una carta que requiere interactividad diferida, el motor genera un objeto `pendingAction` en *Redis* indicando el tipo de acción y el jugador que tiene el foco.
- El turno global queda bloqueado algorítmicamente; cualquier petición de acción que no coincida con el `pendingAction` activo es rechazada automáticamente mediante el lanzamiento de excepciones.
- Cuando el cliente envía la resolución a través del controlador, un despachador dinámico invoca la función específica requerida, aplica las alteraciones de estado, purga la acción pendiente y reactiva el ciclo de vida del turno, tal y como se formaliza en el algoritmo de la Tabla §8.2.

Desviaciones y Retos Técnicos

Durante la implementación surgieron complicaciones al sincronizar las mecánicas asíncronas del servidor con la interfaz visual. Al integrar reglas de negocio estrictas (**RN-03**) y evaluar acciones de impacto inmediato, se originaron bloqueos en la selección de cartas y fallos de renderizado. El obstáculo se superó aplicando un control de flujo riguroso en *React*, aislando la resolución de las promesas asíncronas del ciclo de vida de los modales.

Estado de Cierre

Tras depurar los conflictos de estado entre cliente y servidor, el motor de juego integró con éxito la funcionalidad completa de las cartas de acción y el sistema de turnos asíncronos. Se alcanzó un hito crítico en la jugabilidad del sistema, permitiendo a los jugadores

Identificador	Descripción de la acción de alto nivel
game-001	Resolución Pending Actions
Métodos de alto nivel	
[Object] resolvePendingAction (gameId:String, userId:Int, targetData:Object)	
Pasos (Usar Pseudocódigo o similar)	
Recuperar el estado actual (gameState) almacenado en Redis mediante el gameId.	
Validar que el userId de la petición coincide con el jugador que tiene el foco activo en el objeto pendingAction.	
Invocar al despachador de cartas para aplicar el efecto mecánico sobre el targetData.	
Si la resolución es exitosa, aplicar las mutaciones y purgar el objeto pendingAction del estado general.	
Transicionar el foco y guardar el nuevo estado atómico en Redis.	

Cuadro 8.2: Descripción algorítmica de la resolución de acciones asíncronas en el servidor.

- 1 interactuar con la mesa y dejando el terreno preparado para perfilar las condiciones de
- 2 victoria en el futuro.

3 8.4.4 Iteración 10: Condiciones de Victoria y Resolución de Partida

4 Objetivos de la Iteración

- 5 El objetivo de este ciclo en *King and Peasant* fue definir la finalización algorítmica de
- 6 las partidas, estableciendo las condiciones de victoria asimétricas de cada facción y
- 7 puliendo mecánicas clave de final de juego como el espionaje o infiltración.

8 Desarrollo e Implementación

- 9 El avance principal de este bloque supuso codificar la *lógica de negocio* para comprobar
- 10 de forma asíncrona la victoria del Rey (usando guardias) o del Campesino (exponiendo
- 11 al asesino o mediante revueltas). Dando cumplimiento al requisito de arquitectura
- 12 (**RNF-ARQ**) y garantizando la atomicidad de victoria (**RN-05**), la resolución transac-
- 13 cional de las partidas se orquestó asegurando que la base de datos relacional reflejara
- 14 permanentemente el cambio de estado de manera atómica, bloqueando y limpiando la
- 15 pila de acciones, y coordinándolo con la caché en memoria. En cliente, se implementaron
- 16 modales dinámicos para interactuar y procesar el resultado de la partida.

17 Profundización Técnica: Verificación Algorítmica de la Victoria Asimétrica

- 18 La complejidad de este hito radicaba en la multiplicidad de vías de victoria simultáneas.
- 19 Validar si la partida ha concluido exige procesar estados combinados del tablero tras

cada movimiento, evitando rigurosamente condiciones de carrera donde ambos jugadores pudiesen ser declarados ganadores en la misma fracción de segundo. El flujo de validación se estructuró de la siguiente forma:

- Al finalizar cualquier mutación de estado en *Redis* (ya sea por jugar una carta o vaciar un mazo), se invoca un interceptor central de validación.
- El algoritmo evalúa de forma paralela las condiciones absolutas del Campesino (presencia del Asesino, masa de rebeldes) frente a las condiciones del Rey (límites de guardias, robo de mazo condicionado).
- Si una aserción retorna un estado ganador válido, el motor consolida el estado final en *MariaDB* dentro de una misma transacción de *Prisma*, enviando seguidamente un código de interrupción a los *WebSockets* que destruye el hilo de memoria y renderiza los modales de cierre definitivos en los clientes.

Desviaciones y Retos Técnicos

Durante la transición de finalización de juego se detectaron errores críticos por problemas de concurrencia; en particular, un fallo del servidor (Error 500) al actualizar el estado final y destruir el hilo de la partida. La solución fue encapsular la actualización del usuario y la destrucción del hilo de Redis dentro de un bloque transaccional atómico *try/catch* con *await*.

Estado de Cierre

Las condiciones de victoria y mecánicas finales se estabilizaron con éxito. Todos los componentes se integraron y validaron en la rama *trunk*, permitiendo que el motor asimétrico determinara a un ganador válido de forma autónoma y robusta.

8.5 ASEGURAMIENTO DE CALIDAD Y PULIDO FINAL

Flujo de Interacción y Arquitectura

La estrategia de validación final adopta un enfoque orientado al aseguramiento de la calidad mediante pruebas de integración automáticas. A nivel de infraestructura, se utiliza el motor *Jest* para instanciar el *Game Service* y los servicios de usuarios junto con bases de datos aisladas en memoria. Esto permite simular con precisión el flujo asimétrico de la partida, certificando la invariabilidad algorítmica de las condiciones de victoria y mitigando la aparición de regresiones indeseadas tras refactorizaciones de última hora.

Estructura de Directorios

```

packages/
+-- server/
    +-- tests/
        |-- gameService.test.mjs
        |-- userRoutes.test.mjs
    +-- userService.test.mjs

```

Snippets Estratégicos de Código

```

// packages/server/tests/gameService.test.mjs
describe('gameService', () => {
  test('startGame_inicializa_la_partida_correctamente_con_las_reglas_de_Ki
    // ... logica interna de aserciones y simulacion omitida
  });
});

```

8.5.1 Iteración 11: Interfaces Asíncronas y Modales de Cartas de Acción

Objetivos de la Iteración

El objetivo fundamental de esta iteración consistió en desarrollar la capa visual e interactiva del núcleo del juego, implementando el *frontend* de las Cartas de Acción y el complejo sistema de Acciones Pendientes (*Pending Actions*) para permitir a los jugadores reaccionar a eventos asimétricos.

Desarrollo e Implementación

A lo largo de este *sprint*, los esfuerzos se centraron en traducir la lógica del servidor a una experiencia de usuario fluida en el cliente (*React*). Se construyeron interfaces para interactuar con la mano del jugador y modales dinámicos para explorar la pila de descartes. A nivel arquitectónico y cumpliendo el requisito de rendimiento (**RNF-REN**), se integró el *Game Service* del cliente con la recepción de eventos por *WebSockets*, creando un diccionario visual unificado (`pendingActionsUI.tsx`) capaz de renderizar vistas en base al estado del servidor. Adicionalmente, se programaron las resoluciones asíncronas de cartas de ambos bandos, como el ataque *Strike* del Rey y las emboscadas de los rebeldes del Campesino.

Profundización Técnica: Componentes Reactivos y Diccionario de Acciones Pendientes 1

El reto computacional en la capa de vista consistió en lidiar con interrupciones del flujo en tiempo real. Puesto que las acciones pendientes pausan el curso normal del turno obligando al rival a responder, el cliente debe congelar la interfaz general e inyectar un componente modal específico sin perder el contexto previo. La solución arquitectónica en *React* se diseñó del siguiente modo: 7

- Se implementó un diccionario dinámico de componentes en el archivo `pendingActionsUI.tsx` que mapea claves de acción (*keys*) provenientes del servidor con sus respectivos *Modals*. 9
- Cuando el *Socket.io* notifica un estado que incluye un objeto `pendingAction`, el componente principal de la partida (*Game*) evalúa qué jugador tiene el foco. 11
- Si el foco recae en el jugador local, la vista invoca el componente correspondiente a través del diccionario, forzando la decisión del usuario y bloqueando algorítmicamente el resto de interacciones (*force play*) hasta que el controlador del servidor resuelva la promesa asíncrona. 16

Desviaciones y Retos Técnicos 17

Durante la conexión del diccionario de *Pending Actions* con la vista general, se encontraron repetidos problemas de sincronización de estado y desajustes visuales (*frontend bugs*). Muchos de estos fallos surgieron al intentar que el usuario interactuara con el mazo o las cartas de su mano cuando los modales no se desmontaban correctamente. Esto se solventó reestructurando la jerarquía de dependencias en los *hooks* de *React* y forzando el paso de turno sólo cuando la acción regresaba exitosamente desde el *backend*. 23

Estado de Cierre 24

La integración de los modales y el flujo interactivo de las cartas se estabilizó de forma exitosa. Todo el *frontend* asíncrono se fusionó con la rama *trunk*, proporcionando finalmente a los jugadores la capacidad visual de ejecutar y resolver efectos dinámicos en la partida. 28

8.5.2 Iteración 12: Consolidación Funcional del Motor de Juego y Lógica de Cartas 29

Objetivos de la Iteración 30

El propósito principal establecido para este ciclo consistió en continuar y afianzar la implementación del núcleo de negocio del juego. El esfuerzo se dirigió a consolidar el 32

abánico completo de funcionalidades mecánicas de las cartas de ambos bandos (Rey y Campesino), asegurando que las acciones asimétricas se resolvieran de forma íntegra y transparente tanto en el servidor como en la vista del cliente.

Desarrollo e Implementación

Durante este bloque de trabajo, se completó la lógica subyacente para la gran mayoría del mazo de acción, abarcando desde las tácticas de los guardias del Rey (`kingPendingActions.js`) hasta las mecánicas de revuelta y emboscada de los rebeldes (`peasantPendingActions.js`). Dando cumplimiento al requisito de funcionalidad (RNF-FUN), se programaron rutinas que orquestan las acciones diferidas complejas, alterando de forma segura el estado de las entidades sobre el tablero y calculando daños, escudos y penalizaciones de forma atómica. En el *frontend*, la iteración introdujo nuevos componentes reactivos, destacando la incorporación de un modal dinámico para visualizar e interactuar con el mazo general durante la partida.

Profundización Técnica: Resolución Exhaustiva de Acciones Asimétricas

La materialización algorítmica de la funcionalidad total de las cartas implicó un manejo pormenorizado del flujo de estado. Para aquellas cartas que involucraban selección de objetivos o visualización de áreas (como consultar el mazo o atacar un pueblo), la *lógica de negocio* fue diseñada para bifurcarse:

- El cliente captura la orden inicial y el servidor valida su legalidad contra las reglas asimétricas.
- En caso de requerir interacción múltiple (ej. una carta de rebelde que permite mirar cartas del rival y descartar una), el estado *stateless* transiciona creando el `pendingAction` necesario, actualizando el *GameService* para inyectar al cliente los datos temporales del mazo a examinar.
- La resolución final unifica las propiedades alteradas y reanuda el turno general purgando la caché de la acción diferida, garantizando una ejecución limpia y sin corromper la asimetría de la memoria de *Redis*.

Desviaciones y Retos Técnicos

El aumento en la densidad de reglas de negocio en los controladores de cartas generó fricciones temporales en la comunicación entre las interfaces y la resolución en el *backend*. En concreto, se detectaron fallos lógicos aislados al intentar enlazar el diccionario visual de `pendingActionsUI.tsx` con resoluciones múltiples encadenadas, lo

que obligó a refactorizar las respuestas de *GameService* para homogeneizar los formatos de transferencia (DTO) en cada validación intermedia.

Estado de Cierre

La iteración culminó dejando el sistema de cartas prácticamente completado (a excepción de la mecánica avanzada de infiltración, pospuesta de manera consciente). El núcleo interactivo del juego adquirió una base sólida, garantizando que el flujo central asimétrico fuera enteramente jugable de principio a fin.

8.5.3 Iteración 13: Interfaces de Infiltración y Estabilización del Bucle de Partida

Objetivos de la Iteración

El propósito de este bloque de trabajo consistió en completar la capa de presentación de mecánicas avanzadas, específicamente la interfaz de infiltración, y garantizar la robustez algorítmica del bucle de partida. Esto abarcó la estabilización de la transición entre Eras y la validación determinista de las condiciones de victoria.

Desarrollo e Implementación

El esfuerzo técnico se centró en la mecánica especial de infiltración del Campesino. Al ser una acción que exige examinar temporalmente la mano del oponente, se desarrolló a nivel de *frontend* un componente altamente reactivo (*InfiltrateModal.tsx*) acoplado al diccionario de acciones pendientes, dotando al jugador de una interfaz visual segura y controlada para esta manipulación. En paralelo, y para asegurar la fiabilidad (**RNF-FIA**) del motor, se consolidó la lógica del servidor (*GameService*) responsable de evaluar de forma continua las aserciones de victoria (*Win Condition*) y se estabilizaron los scripts de entorno local (`npm run start:local`) para garantizar un despliegue de desarrollo unificado.

Profundización Técnica: Componentes Reactivos para la Infiltración

La interfaz de Infiltración representó un reto en la capa de la vista debido a su naturaleza asíncrona:

- El componente *React* captura el objeto de transferencia (DTO) efímero enviado por el servidor, inyectando el *InfiltrateModal* sobre el tablero.
- La interfaz congela el entorno general de la partida y fuerza la elección interactiva del usuario sin perder el contexto visual del juego.
- El estado visual se coordina estrictamente con la respuesta del servidor, asegurando que los modales se desmonten de forma limpia y la partida reanude su cauce

1 natural una vez que la validación en *Redis* es exitosa.

2 **Desviaciones y Retos Técnicos**

3 Durante las pruebas del bucle final de juego, se detectaron discrepancias lógicas críticas
4 que corrompían el estado de la mano de los jugadores durante el cambio de fase al invocar
5 `setUpNewEra`. La asignación de cartas no se estaba reiniciando correctamente en
6 memoria. Este bloqueo requirió una reescritura de los ciclos de reasignación y limpieza
7 de la mano del jugador dentro del *GameService*, asegurando que el nuevo ciclo comenzara
8 con un estado íntegro y determinista.

9 **Estado de Cierre**

10 La iteración culminó con la estabilización de los estados límite del juego (transiciones de
11 Era y condiciones de victoria) y la entrega de la interfaz interactiva para la Infiltración.
12 Con un núcleo asimétrico robusto en su bucle final, el proyecto quedó preparado para
13 afrontar las fases conclusivas de pulido general y pruebas de integración.

14 **8.5.4 Iteración 14: Pulido de Reglas, Corrección de Errores y Refactorización Estructural** 15

16 **Objetivos de la Iteración**

17 El propósito central de esta iteración consistió en el pulido exhaustivo de las reglas del
18 juego y el arreglo de errores arrastrados durante las fases de desarrollo previas. Con el
19 núcleo asimétrico ya operativo, el esfuerzo se dirigió a optimizar la mantenibilidad del
20 código (**RNF-MAN**), fortalecer la fiabilidad ante casos límite y dotar al usuario de un
21 *feedback* visual más claro frente a eventos de error.

22 **Desarrollo e Implementación**

23 A nivel de reglas de negocio, se corrigieron desajustes sutiles pero críticos en mecánicas
24 clave, tales como la victoria del Rey mediante el uso de Guardias, las resoluciones de
25 la carta *Revolt* y la exposición lógica del Asesino. En la capa de *frontend*, se introdujo
26 un nuevo sistema de notificaciones asíncronas (`ErrorToast.tsx`) para interceptar y
27 mostrar visualmente al jugador los errores emitidos por el servidor. De forma paralela,
28 y en respuesta a la creciente deuda técnica, se acometió una refactorización masiva del
29 componente principal de la partida (`Game.tsx`).

30 **Profundización Técnica: Desacoplamiento de Lógica y Vista en React**

31 El archivo central de la vista de juego había acumulado una complejidad inmanejable al
32 aglutinar gestión de estado, suscripciones a WebSockets y renderizado condicional de
33 múltiples sub-áreas (mano, tablero rival, modales). La estrategia de refactorización se

apoyó en los siguientes pilares arquitectónicos: 1

- **Custom Hooks:** Se extrajo toda la lógica transaccional de *React* hacia *hooks* especializados (`useGameActions.ts`, `useGameData.ts`), aislando las peticiones asíncronas y el control del flujo del componente visual puro. 2 3 4
- **Descomposición Funcional:** Se fragmentó el árbol de JSX en componentes atómicos con responsabilidades únicas (`PlayerArea.tsx`, `RivalArea.tsx`, `GameSidebar.tsx`) 5
- **Gestión de Modales:** Todas las interfaces de interrupción (`GameOverModal`, `DeckModal`, `PendingActionModal`) fueron encapsuladas y extraídas, aligerando el ciclo de vida del componente raíz y mitigando re-renderizados innecesarios. 8 9

Desviaciones y Retos Técnicos 10

Durante las pruebas de estrés del flujo final de partida, se descubrió un grave error no controlado en el *backend* (Error 500) que ocurría de forma intermitente cuando el juego intentaba concluir y destruir la sesión. Este defecto crítico se solucionó encapsulando el cierre de la partida en el `GameController` dentro de una gestión robusta de excepciones, asegurando la atomicidad en la destrucción de los hilos de *Redis* sin colapsar el servidor. 11 12 13 14 15

Estado de Cierre 16

La iteración resultó en un avance inmenso en términos de calidad de código y solidez funcional. La resolución de fallos arrastrados (condiciones de victoria, desconexiones, cierre de hilo) y la profunda refactorización estructural dejaron una base limpia y altamente estable, preparando el terreno ideal para afrontar las fases finales. 17 18 19 20

8.5.5 Iteración 15: Sistema de Notificaciones Globales y Alertas de Partida 21

Objetivos de la Iteración 22

El propósito de esta iteración consistió en dotar a la plataforma de un sistema global y transversal de *feedback* visual para el usuario. Con el motor asimétrico ya estabilizado tras las refactorizaciones previas, el objetivo técnico fue interceptar eventos asíncronos críticos (como la finalización de una partida o el abandono de salas de espera) y comunicarlos de forma inmediata mediante modales reactivos. 23 24 25 26 27

Desarrollo e Implementación 28

Se diseñó e implementó un sistema de notificaciones transversales en el *frontend* mediante la creación del componente `AnnouncementModal.tsx`. Este nuevo elemento visual se integró en la capa superior de la aplicación, interceptando los estados del ciclo de vida del 29 30 31

juego y de las salas de espera (*Lobbies*). Dando cumplimiento al requisito de usabilidad, se configuraron alertas específicas para notificar a los clientes cuando un jugador decidía abandonar un *Lobby* activo y, fundamentalmente, para presentar la finalización oficial de la partida a todos los implicados de manera sincronizada.

Profundización Técnica: Inyección Reactiva de Modales Globales

El mayor reto arquitectónico radicó en la inyección de este nuevo modal en múltiples vistas sin interferir con la estructura recién refactorizada de `Game.tsx` ni entorpecer el flujo de la sala de espera. La solución consistió en:

- Ampliar el estado global gestionado por los *hooks* personalizados (especialmente `useGameData.ts`) para incluir variables que escucharan de forma activa los eventos de interrupción emitidos por *Socket.io*.
- Acoplar el componente `AnnouncementModal` en la raíz visual de las vistas afectadas (`Game.tsx` y `LobbyRoom.tsx`), manteniéndolo oculto por defecto y delegando su aparición al motor reactivo de estado.
- Garantizar que, al emitirse una notificación crítica (como el fin de juego), la interfaz de usuario bloqueara interacciones residuales y guiara al jugador hacia una transición segura.

Desviaciones y Retos Técnicos

Durante la integración cruzada del `AnnouncementModal`, se experimentaron conflictos de renderizado donde el modal se instanciaba múltiples veces o retenía información obsoleta tras salir abruptamente de un *Lobby*. Estos desajustes visuales (*frontend bugs*) requirieron un ajuste meticuloso en el ciclo de vida de los componentes en *React*, asegurando la limpieza estricta de las suscripciones a los *WebSockets* durante las fases de desmontaje.

Estado de Cierre

La inclusión de estos modales de anuncio enriqueció de forma sustancial la interfaz del cliente, dotando a la aplicación de un canal de comunicación claro para los cierres de sesión y el abandono de salas. Todos los cambios se consolidaron en la rama principal sin mayores incidencias, dando por concluida la implementación del flujo de notificaciones.

8.5.6 Iteración 16: Automatización de Pruebas y Validación Integral del Sistema

Objetivos de la Iteración

El último hito de desarrollo se centró en asegurar de forma definitiva la calidad del

software construido mediante la automatización de pruebas. El esfuerzo técnico buscó blindar tanto la *lógica de negocio* del motor asimétrico como la integridad del módulo de usuarios y las interfaces del cliente, garantizando la fiabilidad (RNF-FIA) del producto antes de su cierre.

Desarrollo e Implementación

A lo largo de esta semana se orquestó una *suite* de pruebas exhaustiva que abarcó todo el *stack* tecnológico. En el *backend*, se implementaron validadores estrictos mediante *middlewares* para las rutas del usuario (`userValidators.js`) y se desarrollaron extensas pruebas de integración tanto para el módulo de cuentas (`userRoutes.test.mjs`) como para el motor del juego (`gameService.test.mjs`, superando las dos mil líneas de acciones). En el *frontend*, se blindaron los flujos críticos mediante pruebas de componentes en *React* (como `Login.test.tsx` o `Profile.test.tsx`), verificando el comportamiento del proveedor de estado frente a la vista.

Profundización Técnica: Resolución de Complejidad en Mecánicas de Cascada

Las pruebas automatizadas sometieron a estrés la resiliencia del motor de juego, logrando exponer la fragilidad oculta en mecánicas complejas como *Revolt* y *Strike*. El motor fallaba intermitentemente al propagar impactos de forma concurrente sobre el tablero asimétrico. La refactorización algorítmica para estabilizar estas acciones garantizó la integridad transaccional en *Redis*:

- Se rediseñó el ciclo lógico dentro de *GameService*, aislando las mutaciones de poder o resistencia en buffers temporales durante el cálculo de daño de área.
- Se implementó una resolución en cascada estricta, asegurando que mecánicas masivas (como el despliegue concurrente de rebeldes en *Revolt*) no sobrescribieran referencias de memoria previas en el ciclo de *JavaScript* sin una validación atómica.
- Esta aproximación garantiza la inmutabilidad: si la resolución asíncrona detecta fallos de interdependencia, la operación es abortada, preservando la partida.

Desviaciones y Retos Técnicos

Al ejecutar las pruebas de integración del *GameService*, el sistema reportó discrepancias graves en el estado en memoria. En concreto, los *tests* evidenciaron que la carta *Strike* no aplicaba correctamente la letalidad programada y que *Revolt* no despachaba la cantidad esperada de tropas. Estos hallazgos detuvieron temporalmente el progreso de la *suite* automatizada, requiriendo reescribir parte de la lógica en los *pendingActions* de los

1 bandos. Se iteró sobre estas funciones hasta que el 100% de los *tests* se resolvieron en
2 verde.

3 **Estado de Cierre**

4 Tras solventar las fricciones mecánicas y alcanzar una tasa de éxito íntegra en los entornos
5 de prueba (*frontend* y *backend*), el proyecto quedó enteramente estabilizado. Esta última
6 iteración cerró de forma exitosa el ciclo de implementación técnica en el repositorio,
7 entregando un producto robusto, validado empíricamente y listo para su defensa.

PRUEBAS Y VALIDACIÓN DEL SISTEMA

En este capítulo se describe la estrategia de verificación adoptada para asegurar la calidad y el correcto funcionamiento de la plataforma *King and Peasants*. Se detalla el diseño de las pruebas siguiendo una arquitectura de pirámide, la validación de los componentes de interfaz, la integridad de la API y el motor de lógica asimétrica, así como el flujo de integración continua que garantiza la estabilidad en cada despliegue.

9.1 ESTRATEGIA DE PRUEBAS Y ENTORNO

Se ha diseñado una estrategia de pruebas integral que abarca desde la lógica de negocio más granular hasta el comportamiento de la interfaz de usuario. El stack tecnológico de pruebas se divide según la naturaleza de cada paquete:

- **Cliente (Frontend):** Se utiliza **Vitest** como motor de ejecución de pruebas por su integración nativa con Vite, junto con **React Testing Library (RTL)** para la validación de componentes basada en el comportamiento del usuario.
- **Servidor (Backend):** Se emplea **Jest** para la ejecución de pruebas unitarias y de integración, complementado con **Supertest** para la realización de peticiones HTTP sintéticas contra los *endpoints* de la API.

La pirámide de pruebas implementada asegura que la base del sistema esté cubierta por pruebas unitarias rápidas y ligeras, mientras que las capas superiores validan la integración de servicios (API y base de datos) y la interactividad de la interfaz.

9.1.1 Validación del Cliente (Frontend)

La validación del frontend se centra en dos pilares: la correcta renderización de componentes y la gestión del estado global mediante la *Context API*.

Pruebas de Componentes y Vistas

Se evalúa el montaje del árbol DOM virtual y la captura de eventos. Por ejemplo, en la vista de inicio de sesión (`Login.tsx`), se verifica que los campos de entrada capturen la información y que el flujo de navegación se active ante una respuesta exitosa del servidor. El siguiente fragmento ilustra cómo se utiliza `user-event` para simular la interacción:

Listing 9.1: Prueba unitaria del componente de Login

```
test('hace_login_y_navega_a_home_cuando_la_API_responde_OK', async () => {
  const fetchMock = vi.mocked(fetch);
  fetchMock.mockResolvedValue({
    ok: true,
    json: async () => ({
      userId: 1, name: 'John', email: 'john@test.com', authToken: 'token
    }),
  } as Response);

  render(<MemoryRouter><Login /></MemoryRouter>);

  await userEvent.type(screen.getByPlaceholderText('Email'), 'john@test.
  await userEvent.type(screen.getByPlaceholderText('Password'), 'secret'
  await userEvent.click(screen.getByRole('button', { name: 'Login' }));

  await waitFor(() => {
    expect(mockLogin).toHaveBeenCalledWith({
      id: 1, name: 'John', email: 'john@test.com', authToken: 'token-123
    });
    expect(mockNavigate).toHaveBeenCalledWith('/');
  });
});
```

Pruebas de Estado (Context API)

Se valida la lógica de persistencia y protección de rutas en el `AuthProvider`. Se comprueba que, tras un inicio de sesión exitoso, el estado de la aplicación se actualice, se almacene el token en el *local storage* y se inicialice la conexión por *sockets* para la comunicación en tiempo real.

9.1.2 Validación del Servidor (Backend)

El backend se valida mediante pruebas de integración que aseguran la consistencia de los datos y el cumplimiento de las restricciones de seguridad.

Pruebas de Integración de API (Endpoints)

Se utiliza Supertest para validar que los controladores responden con los códigos de estado HTTP adecuados (200 OK, 400 Bad Request, 401 Unauthorized, etc.). Se pone especial énfasis en la validación de esquemas de entrada, asegurando que el sistema rechace peticiones con datos malformados.

Listing 9.2: Validación de endpoints con Supertest

```
test('POST /api/auth/register devuelve 400 con payload invalido', async () {
  const app = buildApp();
  const response = await request(app)
    .post('/api/auth/register')
    .send({ name: 'ab', email: 'bad-email', password: 'short' });

  expect(response.status).toBe(400);
  expect(response.body.message).toBe('Validation_error');
  expect(response.body.errors).toEqual(expect.arrayContaining([
    expect.objectContaining({ message: 'Name_must_be_at_least_3_characters' }),
    expect.objectContaining({ message: 'Email_must_be_a_valid_email_address' })
  ]));
});
```

Pruebas Transaccionales y de Servicios

Se aíslan los servicios de lógica de usuario (UserService) y gestión de salas (LobbyService) mockeando la capa de persistencia (Prisma) cuando es necesario o utilizando bases de datos de prueba para asegurar que las mutaciones cumplen el modelo relacional definido.

9.1.3 Validación del Motor Asimétrico (Game Service)

Esta sección representa el núcleo técnico más complejo. El servicio GameService gestiona el estado atómico de las partidas almacenado en Redis y aplica las reglas asimétricas del juego.

Metodología de Validación

1

Se emplea el patrón **Arrange-Act-Assert** (Preparar-Actuar-Verificar):

2

1. **Arrange:** Se prepara un estado de partida en memoria (mock de Redis) con una configuración específica de cartas en mano, pueblo y mazo.
2. **Act:** Se ejecuta una acción de juego (jugar carta, resolver habilidad).
3. **Assert:** Se comprueba que el nuevo estado devuelto cumple con las Reglas de Negocio (RN), incluyendo el cambio de turno y la visibilidad selectiva de las cartas (lo que un jugador puede ver frente a lo que el otro desconoce).

3

4

5

6

7

8

Verificación de Mecánicas Complejas

9

Un ejemplo crítico es la validación de la condición de victoria mediante el Guardian revelando al Assassin en el mazo. Esta prueba asegura que la lógica de fin de partida se dispare correctamente y se actualicen los registros en la base de datos relacional de forma atómica.

10

11

12

13

Listing 9.3: Prueba de lógica asimétrica de victoria

```
test('Guardian revela al Assassin en el mazo y cierra la partida', async (
  const guardian = makeCard({ uid: 'g1', templateId: 7, typeKing: 'Guard
  const assassin = makeCard({ uid: 'a1', templateId: 16, typePeasant: 'R

  await saveState('game-1', createState({
    turn: 'king', deck: [assassin],
    players: { king: { town: [guardian] } },
  }));

  const result = await gameService.resolvePendingAction('game-1', 1, {
    targetUid: 'a1',
  });

  expect(result).toEqual({
    isGameOver: true,
    winnerId: 1,
    reason: 'ASSASSIN_EXPOSED',
  });
```

14

15

16

17

18

19

20

21

22

23

24

25

26

27

28

29

30

31

```

1      expect(prismaMock.game.create).toHaveBeenCalledTimes(1);
2    });

```

9.2 INTEGRACIÓN CONTINUA Y DESPLIEGUE (CI/CD)

Para garantizar que ningún cambio en el código degrade la calidad del sistema, se ha implementado un flujo de integración continua utilizando **GitHub Actions**. El pipeline definido en `CD_Deploy.yml` actúa como una barrera de seguridad (*Quality Gate*).

Cada vez que se realiza un *push* a las ramas `main` o `trunk`, se dispara automáticamente un *job* de pruebas que realiza las siguientes acciones:

1. Preparación del entorno con Node.js 20.
2. Instalación de dependencias de todos los paquetes.
3. Ejecución de la suite completa de pruebas (`npm run test`).

Si alguna aserción falla, el pipeline se detiene inmediatamente, bloqueando el despliegue automático hacia Render. Esto asegura que solo las versiones de código que superan satisfactoriamente todas las pruebas de validación lleguen a los entornos de pre-producción y producción, protegiendo así la integridad de la rama principal.

PARTE IV

CIERRE DEL PROYECTO

MANUAL DE USUARIO

The good parts of a book may be only something a writer is lucky enough to overhear or it may be the wreck of his whole damn life – and one is as good as the other.

*Ernest Hemingway (1899–1961),
Novelist*

*R*esumen de lo que va a ocurrir en el capítulo. ¿Cuál es el objetivo que tenemos con este capítulo?

10.1	ÍNDICE	1
1.	Pantalla Inicial	2
2.	Registro/Inicio de Sesión	3
3.	Amistad	4
4.	Listado de Salas	5
5.	Salas	6
6.	Partida	7
10.2	PANTALLA INICIAL	8

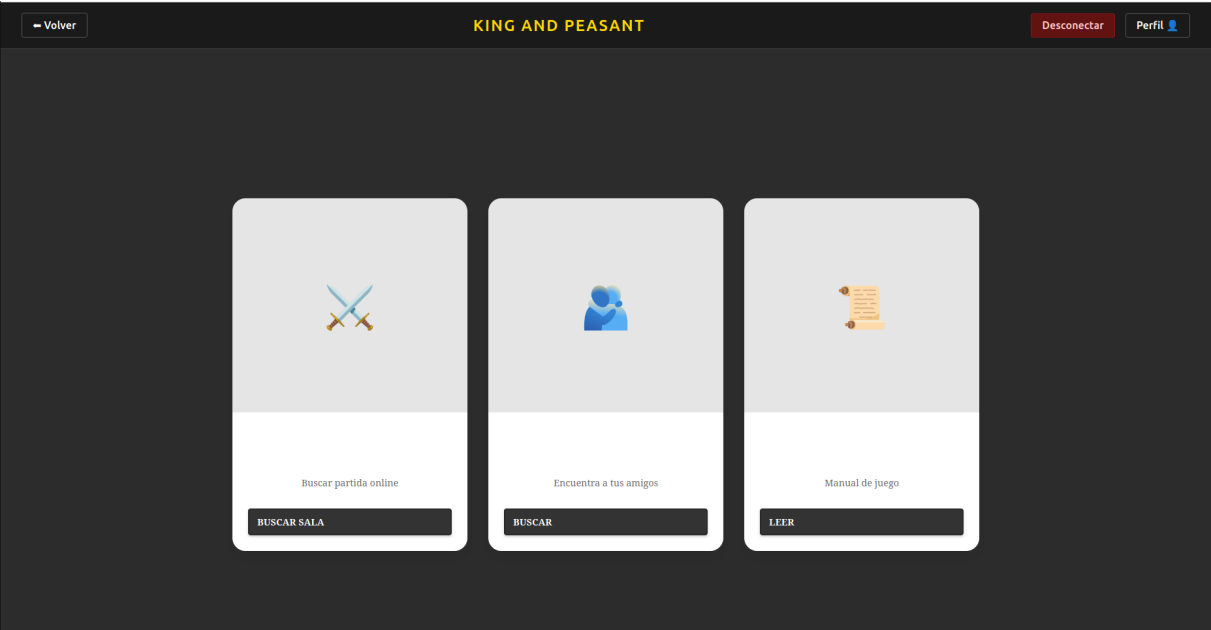


Figura 10.1: Pantalla Inicio

Al entrar en la aplicación nos encontramos con la pantalla de inicio. Desde esta accedemos a las distintas duncionalidades de la aplicación.

1 Pero antes de comenzar a probar la app, debemos iniciar sesión o registrarnos. Para
2 ello accederemos a la pantalla de Registro / Inicio de Sesión desde los botones de la
3 cabecera de la pantalla que se encuentran a la derecha.

4 10.3 REGISTRO/INICIO DE SESIÓN

5 Para registrarnos debemos introducir un nombre, un correo válido y una contraseña
6 segura.

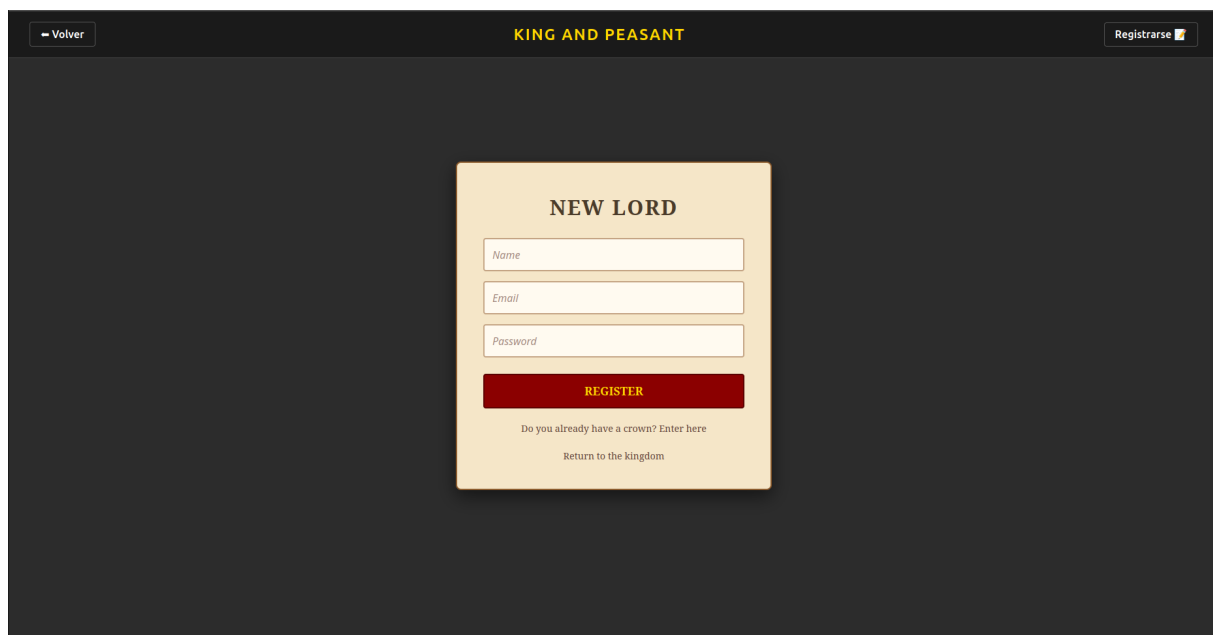
The screenshot shows a mobile application interface with a dark grey background. At the top, there is a header bar with a dark grey background. On the left of the header is a button labeled 'Volver' with a left-pointing arrow. In the center of the header is the text 'KING AND PEASANT' in yellow. On the right of the header is a button labeled 'Registrarse' with a crown icon. Below the header, centered on the screen, is a light yellow rectangular form titled 'NEW LORD'. Inside the form, there are three input fields labeled 'Name', 'Email', and 'Password'. Below these fields is a red button with the word 'REGISTER' in yellow. At the bottom of the form, there is a link that says 'Do you already have a crown? Enter here' followed by 'Return to the kingdom'.

Figura 10.2: Pantalla de Registro

7 Una vez registrados podemos iniciar sesión introduciendo nuestro correo y nuestra
8 contraseña.

← Volver

KING AND PEASANT

Registrarse

**LOGIN TO THE
KINGDOM**

Email

Password

LOGIN

Don't you have a crown yet? [Register here](#)

[Return to the kingdom](#)

Figura 10.3: Pantalla de Inicio de Sesión

1 10.4 AMISTAD

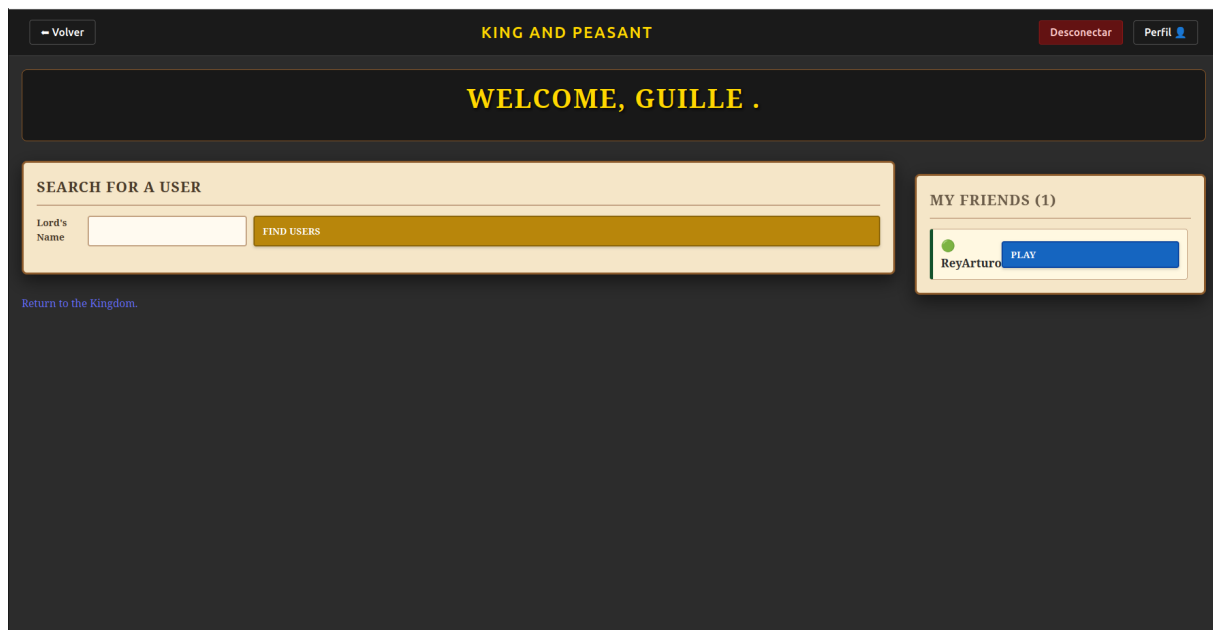


Figura 10.4: Pantalla de Amistad

- 2 Desde aquí podemos, buscar usuarios por su nombre, así como mandarles una
- 3 invitación de amistad.
- 4 En la parte derecha de la pantalla podemos ver los amigos que tenemos.

10.5 LISTADO DE SALAS

1

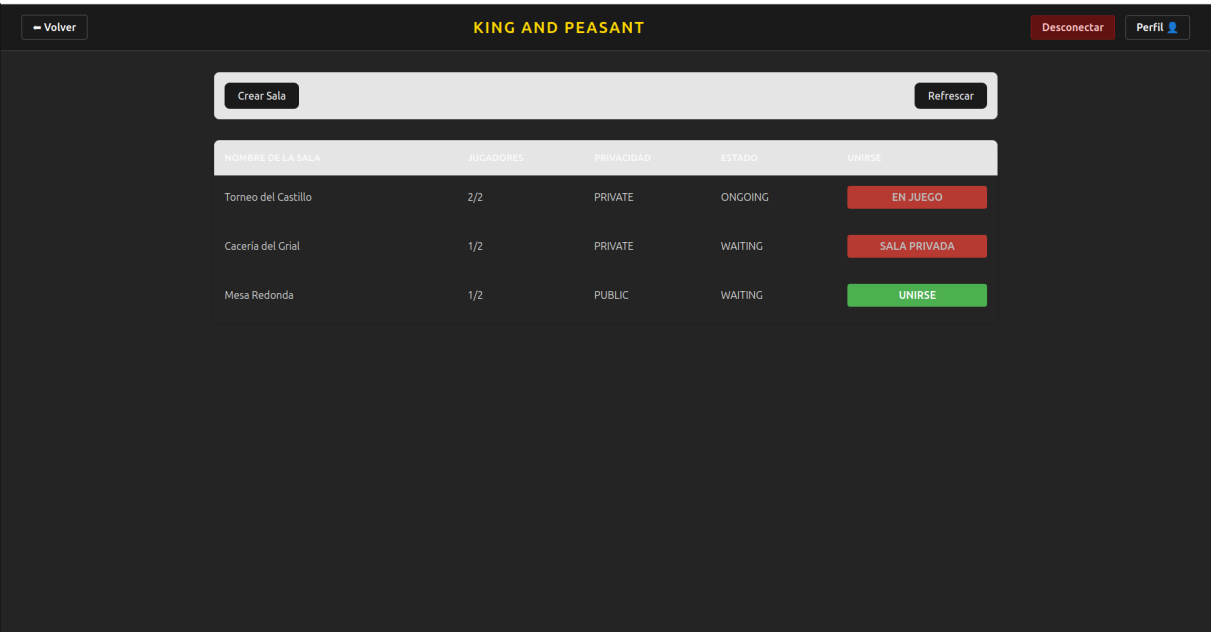


Figura 10.5: Pantalla de Listado de Salas

Ahora desde la página de inicio podemos acceder a la pantalla de Listado de Salas. Desde esta pantalla se puede observar una lista de todas las salas creadas. Podemos ver quién es el creador y el estado de la sala.

Desde aquí podemos realizar lo siguiente:

1 10.5.1 Crear Sala

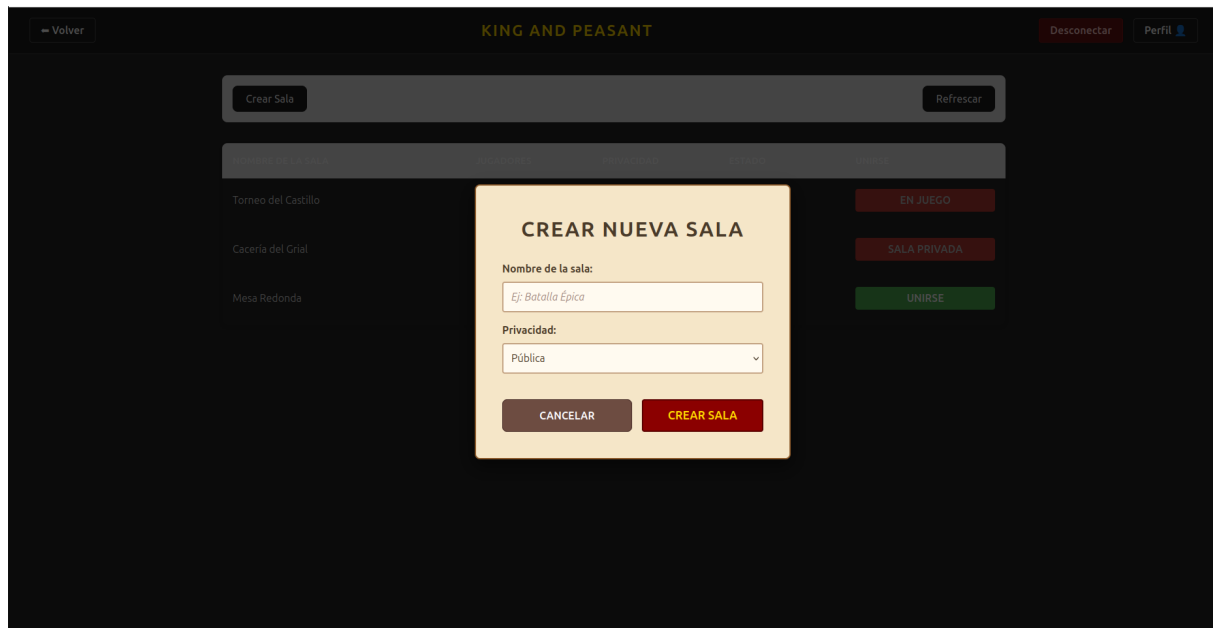


Figura 10.6: Pantalla de Listado de Salas: Crear Sala

- 2 Si pulsamos el botón de Crear Sala podremos crear una sala añadiendo un nombre a
3 esta y especificando si esta es pública o privada.

4 10.5.2 Unirse a una Sala:

- 5 Si no estamos dentro de ninguna sala podremos unirnos a salas públicas que no
6 estén llenas.

10.5.3 Volver a la sala:

1

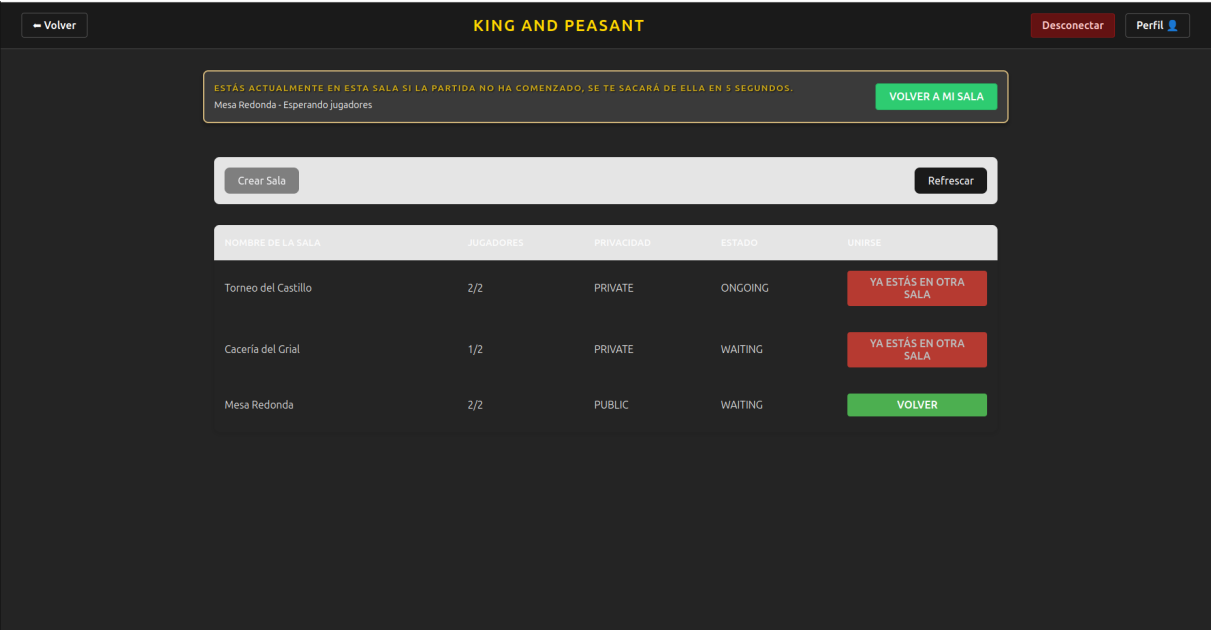


Figura 10.7: Pantalla de Listado de Salas: Volver a la Sala

Si ya estamos en una sala, aparecerá un recuadro con un botón que no redirigirá a esta.

2
3

1 10.6 SALA

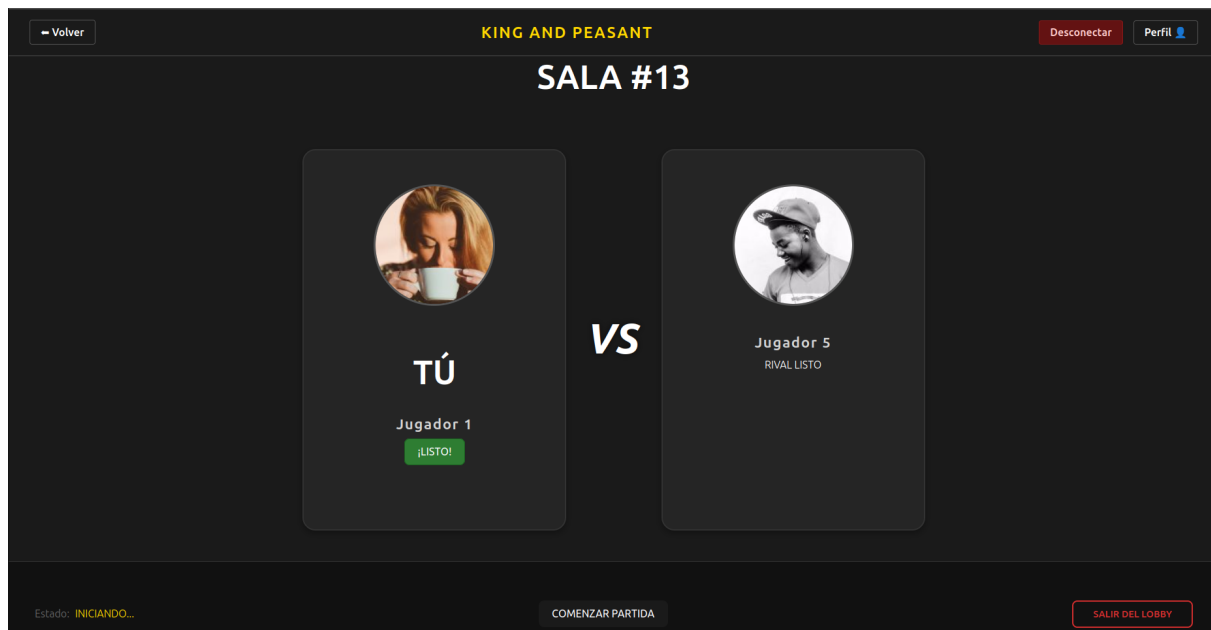


Figura 10.8: Pantalla de Sala

2 Una vez nos encontramos en la sala, debemos marcar el botón de listo para comenzar
3 a jugar. Una vez los dos jugadores en la sala estén listos, el jugador líder podrá iniciar la
4 partida pulsando el botón de Comenzar Partida.

5 Por otro lado, si pulsamos el botón de abandonar sala, saldremos de esta y se nos
6 redirigirá al Listado de Salas.

7 En caso de que salgamos de esta pantalla accidentalmente, desde la pantalla de
8 Listado de Salas no podremos reconectar durante intervalo de 5 segundos desde que
9 salimos de esta pantalla.

10 10.7 PARTIDA

11 10.7.1 Interfaz del Juego

12 Tu Área:

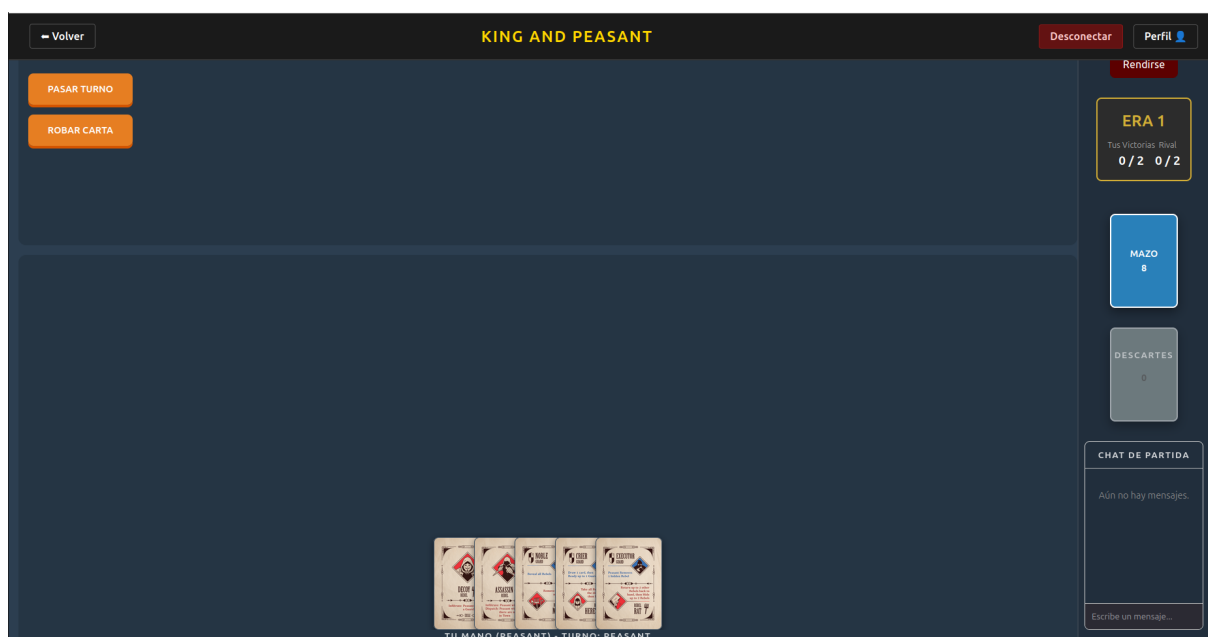


Figura 10.9: Pantalla de Partida: Tu Área

En la parte inferior verás las cartas que tienes en tu mano. Un poco más arriba se encuentra tu pueblo. Desde aquí seleccionarás qué cartas jugar.

Área del Rival:

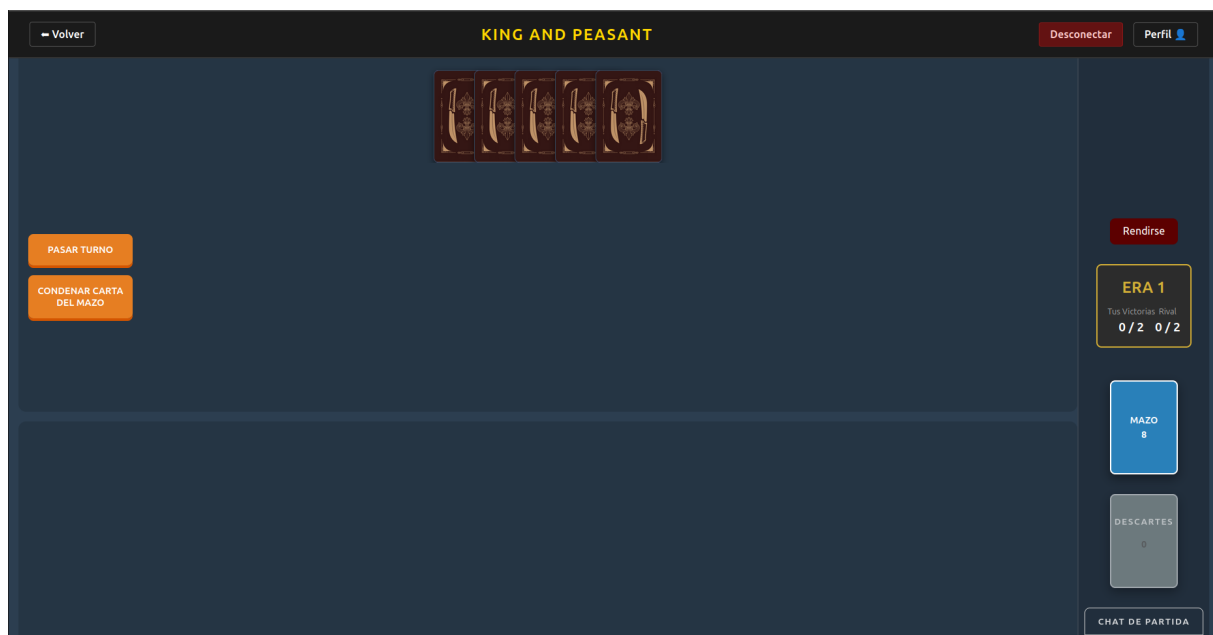


Figura 10.10: Pantalla de Partida: Área del Rival

En la parte superior verás las cartas en la mano de tu rival, estas estarán boca abajo. Un poco más abajo, se encuentra el pueblo rival, con las cartas que este ha colocado, si estas están reveladas aparecerán boca arriba.

Mazo:

Desde donde se roban las cartas. Puedes pulsar en él para interactuar según lo permitan las reglas.

Pila de Descartes:

Donde van las cartas usadas. Puedes hacer clic para inspeccionar las cartas descartadas.

Chat: A un lado de la pantalla tendrás un chat en vivo para hablar con tu oponente mientras jugáis.

10.7.2 Acciones de Turno

Durante tu turno, podrás realizar diversas acciones dependiendo de tu rol (Rey, Campesino):

Robar Carta:	1
Robar una carta del mazo.	2
Colocar Carta en Pueblo:	3
Selecciona una carta de tipo Guard o Rebel para colocar en el pueblo.	4
Jugar Carta en el Pueblo:	5
Selecciona una carta del pueblo para jugar su acción. En caso de que tu rol sea el de Campesino, solo puedes jugar cartas no reveladas.	6
Si la carta se puede infiltrar, deberás seleccionarla y elegir la posición del mazo donde quieres infiltrar la carta.	7
Jugar Acción:	8
Si la carta es de tipo ACTION, esta se juega desde la mano sin necesidad de colocarla en el pueblo.	9
Pasar Turno:	10
Pasar al turno rival sin realizar ninguna acción.	11
Condenar Rebelde:	12
Si tu rol es el rey, tienes la posibilidad de condenar a un rebelde no revelado, ya sea pulsando sobre una carta del pueblo rival o pulsando el botón de Condenar Carta del Mazo, este último condenaría la carta de más arriba del mazo.	13
CUIDADO: Esta acción termina en victoria o derrota en función de si la carta seleccionada era el Asesino	14
Acciones Pendientes:	15
Algunas cartas jugadas por el rival requerirán una respuesta tuya (por ejemplo, seleccionar una carta). La pantalla te pedirá que elijas cómo reaccionar antes de continuar.	16
10.8 FIN DE LA ERA Y FIN DE LA PARTIDA	17
Cuando se cumplan las condiciones de victoria de alguno de los roles, aparecerá una pantalla anunciando el ganador, un resumen rápido y deberás pulsar el botón para continuar con la siguiente ronda/era.	18
Cuando un jugador a ganado dos eras, se acaba la partida y se te redirige a la sala.	19

CONCLUSIONES

The good parts of a book may be only something a writer is lucky enough to overhear or it may be the wreck of his whole damn life – and one is as good as the other.

*Ernest Hemingway (1899–1961),
Novelist*

*R*esumen de lo que va a ocurrir en el capítulo. ¿Cuál es el objetivo que tenemos con este capítulo?

11.1 INFORME POST-MORTEM	1
Qué es un informe post-mortem	2
11.1.1 Lo que ha ido bien	3
■ Argumento a favor 1.	4
■ Argumento a favor 2.	5
■ Argumento a favor 3.	6
11.1.2 Lo que ha ido mal	7
■ Argumento en contra 1.	8
■ Argumento en contra 2.	9
■ Argumento en contra 3.	10
11.1.3 Discusión	11
En función de lo anterior, qué cambiaría si empezara hoy el proyecto de nuevo.	12
11.2 TRABAJOS FUTUROS	13
Enumera los puntos abiertos y que no se han resuelto. Indica si darían lugar a otro proyecto y de qué forma se podría acotar.	14
	15

PARTE V

APPENDICES

SOFTWARE PRODUCT LINES

1

2 *The good parts of a book may be only something a writer is lucky enough to overhear or it may be the wreck*
3 *of his whole damn life – and one is as good as the other.*

4

Ernest Hemingway (1899–1961),

5

Novelist

6

7 **T**his is an example of an abstract. Multiple lines are supported. Several paragraphs. It
8 jumps to the next page. Blau blau blau. I am introducing more text to reach the third
9 line

A.1 SOFTWARE PRODUCT LINES

- Objective of a *Product Line* (PL) (mass production and customisation) [3] 2
- The focus in software derives in *Software Product Lines* (SPLs). 3
- Variability management: variability models 4
- When and how are used VMs: FMs are described in FODA report as a key element in SPL since they represent the variability and commonality of the different products in a SPL. 5
6
7

A.2 FEATURE MODELS

To Abductive Section in 2.1

As the number of products to be built by a SPL may be large and the constraints among features may be complex, representing such an information in a manageable and compact manner is a must. *Feature Models* (FMs) represent the set of products a SPL may build in terms of product features. Some features are optional while others are mandatory. To indicate the relationships among features, they are hierarchically linked, forming a tree whose root is a feature representing the whole functionality of a product. The root feature is refined in child features, which increase the level of detail and reduce the scope of features. Recursively following this refinement process, a tree-like structure is obtained where three basic kinds of hierarchical relationships are used: 10
11
12
13
14
15
16
17
18

- Mandatory: a mandatory relationship affects a parent and child feature. It forces the child feature to appear in a product whenever its parent feature does. 19
20
- Optional: a child feature connected to a parent feature by means of an optional relationship may be optionally selected whenever its parent feature is. 21
22
- Set-relationships: three or more features are part of a set-relationship: a parent feature and a set of two or more child features. A set-relationship contains a cardinality that constraints the number of child features to be selected in a product whenever its parent feature is selected. If the cardinality is [1..1] it is commonly remarked as an *alternative relationship* where only one child feature may be selected at the same time. If the cardinality is [1..N] (where N is the number of child 23
24
25
26
27
28

features), it is also known as an *or-relationship* as any combination of child features is allowed while at least one is selected.

Although FMs can represent most of the most frequent constraints, the hierarchical nature of these models might hinder the representation of some constraints. Under this circumstance, *cross-tree constraints* can be added. The most common kinds of cross-tree constraints are:

- Dependency: a feature depends on another feature if the second one must be part of a product whenever first one is selected.
- Exclusion: two features exclude themselves if both of them cannot be part of a product at the same time.

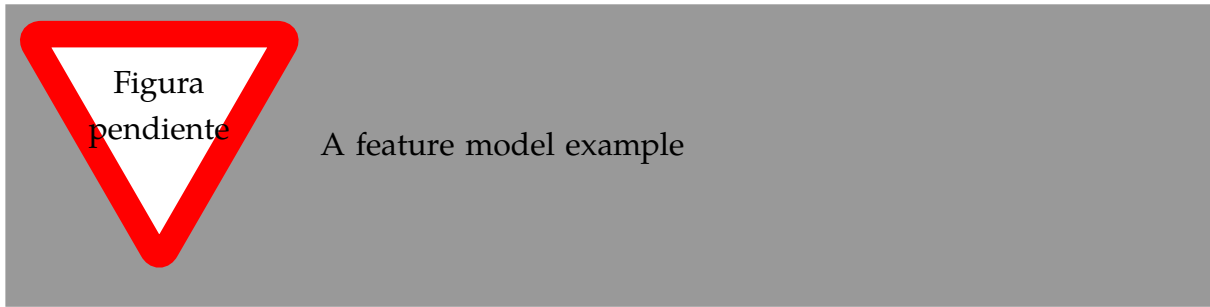


Figura A.1: An example of a Home Integration System

The example in Figure §A.1 describes a *Home Integration System* (HIS) SPL in terms of its features and the relationships among them. Leaning on this example we define some useful terms:

Partial configuration : a partial configuration is a composed by three sets of selected (S), removed (R) and undecided (U) features. A feature can only be in one of these sets and every feature in the FM (fm) must be in one of them, i.e. $S \cup R \cup U = fm$ and $S \cap R \cap U = \emptyset$. A partial configuration represents an intermediate state during the process of a customer selecting the feature for a custom product. For example, $S_P = \{...\}$, $R_P = \{...\}$ and $U_P = \{...\}$ define a partial configuration for the sample FM where some features are still to be decided if they are to be selected or removed in a configuration.

(Full) configuration : a full configuration or simply a configuration is a partial configuration such that the set of undecided features is empty. For example, $S_F = \{...\}$ and $R_F = \{...\}$ describe a full configuration for the example FM.

Product : a product is a representation for a full configuration such that only the selected features are remarked. For instance, $P = \{\}$ is a product for the above full configuration. A product such as A,B is a valid since all the constraints within the FM are satisfied. However, A,B and C is not a valid product since D is required.

Validation A partial configuration is *valid* if all the relationships and constraints are satisfied given the sets of selected, removed and undecided features. So the definition applies for valid full configurations and valid products. As a conclusion we can affirm that a FM represents all the valid products in a SPL.

Objetivo: Briefly expose attributes as an important asset in feature models.

It is frequent that features are not enough to represent information that is relevant to represent a SPL variability. In this case, FMs are extended with feature attributes such as cost, versions, RAM consumption, etc. in the so-called *Extended Feature Models* (EFMs) [3]. Besides relationships, an EFM contains constraints that affect attributes which reduce even more the set of products a FM describes. Above definitions remain when attributes are introduced into FMs.

A.3 AUTOMATED ANALYSIS OF FEATURE MODELS

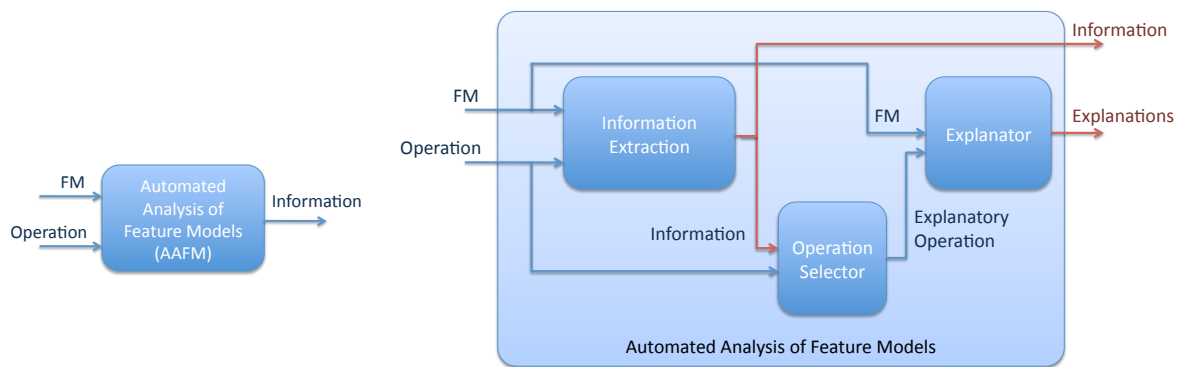
A.3.1 Scope

To Abductive Intro

FMs are used all along the SPL development as key models and many of the development decisions are taken relying on the information contained within them. Most of the times, relationships are complex and hinder the manual extraction of information. Manually obtaining information such as 'which is the product that costs the less?', 'does the feature model contain errors?' or 'why there exist no product containing certain features?' can be an unfeasible task. The complexity and compactness of FMs justify the need of an automated support of these operations. So the *Automated Analysis of Feature Models* (AAFM) arises as a topic of interest to deal with this problem in the SPL community.

The AAFM can be seen as a black-box process that receives a FM and an operation as inputs and obtains information (its kind depends on the analysis operation) as an

output (Fig. A.2(a)). There are many operations that extract information from a FM such as 'counting products' operation whose result is a natural number indicating the number of customised products that can be built; or 'list of products' operation that obtains each of those products. This vision of AAFM as a black-box is valid for a subset of analysis operations that we call *information extraction operations* (IEO) that can be seen as processes to extract information from FMs. In other words, an IEO makes explicit an implicit information within a FM.



(a) The AAFM seen as a black-box process

(b) Extending the AAFM process with explanations

Figura A.2: A different view on AAFM distinguishing between information extraction and explanatory operations

Use me to explain in a larger text than 'sidetext' anything that is important to a reader not familiar with the dissertation context for example.

However, there is a subset of analysis operations known as *explanatory operations* (EO) whose objective is explaining the result obtained from a IEO. Sometimes, the result is not the expected one and the analyser needs to know which are the relationships that have caused it. For example, let us suppose that the IEO 'which are the products described in a FM that cost less than \$1000?' obtains no products as a result. If we were expecting to obtain at least one product, it is important to determine the relationships in the FM that are responsible of that behaviour, so an EO 'why there is no product costing less than \$1000?' will shed light on the relationships that avoid obtaining any product. Obtaining

no result is not the only case that claims for explanations. If we obtained only one product as a result and we were expecting to obtain at least 10 products, although an answer is obtained the result is unexpected and the discrepancy reasons have to be found. Moreover, explanatory operations are also useful even when an expected result

is obtained, to reinforce the certainty that the result is correct. So it can be concluded that EOs complement the information an FM analyser obtains from IEOs.

The complexity of feature modelling relies on correctly setting the relationships that describe the set of products to be built in a SPL. Relationships are the only elements responsible of the results obtained in FM analysis. So an *explanation* is a set of relationships that may have caused that result. While IEO provides for an unique response that is known for certain, an EO provides for a set of probable explanations to a result obtained from a IEO, being only one of them a valid explanation. It would be the analyser the one in charge of discriminating the correct explanation, maybe performing new analysis operations.

**THIS IS A SIDE TEXT. USE TO
REMARK IMPORTANT
INFORMATION**

Therefore, two kinds of operations are distinguished in AAFM: information extraction and explanatory operations. Explanatory operations have no sense without a paired information extraction operation and its result. To ensure that explanatory operations are always paired to an information extraction operation, we define a new black-box process of AAFM that incorporates explanations as an additional output (see Figure A.2(b))

1. Information extraction: the original process, which remains the same.
2. Operation selector: depending on the information extraction operation the analyser asks for and the information obtained as a result, this process provides the explanatory operation to be performed. In other words, it pairs an explanatory operation to an information extraction operation.
3. Explanatory analysis: provides a set of explanations from the FM and the explanatory operation.

The overall process can be encapsulated into a holistic black-box process which receives the FM and the information extraction operation as inputs and provides a result and explanations as outputs. It can be seen as we just add explanations as an output to the analysis process.

To realise this view on the AAFM, we need to give details on the insides of these black-boxes. Since the information extraction process is already rigourously defined in Benavides' PhD dissertation, the purpose of this paper is defining the remaining two

sub-processes. We formalise the explanatory analysis process by means of default logic and provide the criteria to implement the operation selector process.

Most Common Techniques to perform AAFM Operations.

A.4 DYNAMIC SOFTWARE PRODUCT LINES (DSPL)

What is a *Dynamic Software Product Line (DSPL)*. Different points of view. What is important is the automation of reconfiguration properties relying on SPL techniques.

We focus in the application of explanations in DSPLs as an application of our results. Specifically we have worked in MAS and smart homes providing a solution for automating product reconfiguration.

A.5 HYPOTHESIS AND OBJECTIVES

Objetivo: Justifying that explanations are a particular set of operations in AAFM that are not solvable by means of the techniques that are used up-to-date

Objetivo: Set an impacting phrase that summarises the hypothesis

Hypothesis

*Explanations cannot be solved by AI techniques used to solve AAFM.
There should exist other AI techniques to solve explanations.*

Objective of the dissertation

Defining a framework to provide solutions for explanatory analysis in FMs.

This dissertation summarises our contribution to solve some of the objectives we set in our PhD project.

- Defining a catalog of analysis operations where explanations are applied.
- Rigorously defining these operations in terms of logics.
- Proposing solutions to these operations.
- Validating our results by means of tools and projects where they are applied.

Next chapter focuses on refining how we have contributed to deal with the above objectives.

A piece of code...

```
public Map<Cardinality, CardinalValue> detectWrongCardinals() {
    // any other implementation of Map can be used instead.
    Map<Cardinality, CardinalValue> result =
        new TreeMap<Cardinality, CardinalValue>();
    for( r : relationships) {
        if (r instanceof Set) {
            Set set = (Set)r;
            Cardinality card = set.getCardinality();
            Domain dom = card.getDomain();
            for (value: dom.getValues())
                if (isWrongCardinal(card, value))
                    result.put(card, value);
        }
    }
    return result;
}
```

A coolTable. Use inside a table.

Use `\TableSubtitle{n,title}` to add a subtitle as the header. n is the number of columns and title is the text to place. [3]

A Catalog of FM Explanatory Operations (2009 version)		
Information Extraction Operation	FM Explanatory Operations	
	<i>Why? operation</i>	<i>Why not? operation</i>
Valid FM	-	invalid FM
Valid Configuration	valid partial conf.	invalid partial conf.
Valid Product	valid product	invalid product
Products Listing	vaild Product/Config	invalid FM/Product/Config
Products Counting	vaild Product/Config	invalid FM/Product/Config
Optimisation	vaild Product/Config	invalid FM/Product/Config
Core feature	core feature	core feature
Variant feature	variant feature	variant feature
Dead feature detection	-	dead feature
False-optional feature detection	-	false-optional feature
Wrong-cardinality detection	-	wrong cardinal
Information Extraction Operation	Configuration Explanatory Operations	
	<i>Why? operation</i>	<i>Why not? operation</i>
Valid Configuration	valid partial conf.	invalid partial conf.

Cuadro A.1: Most frequently used explanatory operations and their corresponding information extraction operations

BIBLIOGRAFÍA

- [1] Adobe Systems. Flash & the future of interactive content. <https://theblog.adobe.com/flash-the-future-of-interactive-content>, 2017. Anuncio oficial del fin de vida (EOL) de Flash Player. Accedido: 2026-02-18.
- [2] A. Banks and E. Porcello. *Learning React: Modern Patterns for Developing React Apps*. O'Reilly Media, Inc., 2nd edition, 2020. ISBN 9781492051225.
- [3] D. Benavides, A. Ruiz-Cortés, and P. Trinidad. Automated reasoning on feature models. *LNCS, Advanced Information Systems Engineering: 17th International Conference, CAiSE 2005*, 3520:491–503, 2005. ISSN 0302-9743.
- [4] M. Cantelon, M. Harter, T. J. Holowaychuk, and N. Rajlich. *Node.js in Action*. Manning Publications Co., 2013. ISBN 9781617290572.
- [5] J. L. Carlson. *Redis in Action*. Manning Publications Co., 2013. ISBN 978-1617290855.
- [6] G. S. Elias, R. Garfield, and K. R. Gutschera. *Characteristics of Games*. MIT Press, 2012. ISBN 978-0262017138.
- [7] R. Elmasri and S. B. Navathe. *Fundamentals of Database Systems*. Pearson, 7th edition, 2015. ISBN 978-0133970777.
- [8] I. Fette and A. Melnikov. The websocket protocol. RFC 6455, RFC Editor, 2011. URL <https://www.rfc-editor.org/info/rfc6455>.
- [9] S. Loreto, P. Saint-Andre, S. Salsano, and G. Wilkins. Known issues and best practices for the use of long polling and streaming in webRTC, xmlhttprequest and server-sent events. RFC 6202, RFC Editor, 2011. URL <https://www.rfc-editor.org/info/rfc6202>.
- [10] A. Mardan. *React Quickly: Painless web apps with React, JSX, Redux, and GraphQL*. Manning Publications Co., 2017. ISBN 9781617293344.
- [11] G. J. Myers, C. Sandler, and T. Badgett. *The Art of Software Testing*. John Wiley & Sons, 3rd edition, 2011. ISBN 978-1118031964.

- [12] nPipsGames. King and peasant, 2024. URL <https://www.npipsgames.com/king-peasant>. Accedido: 28 de abril de 2024.
- [13] S. Tilkov and S. Vinoski. Node.js: Using javascript to build high-performance network programs. *IEEE Internet Computing*, 14(6):80–83, 2010. doi: 10.1109/MIC.2010.145.
- [14] J. Turnbull. *The Docker Book: Containerization is the new virtualization*. James Turnbull, 2014. ISBN 978-0988820203.
- [15] S. Woods. *Eurogames: The Design, Culture and Play of Modern European Board Games*. McFarland, 2012. ISBN 978-0786467976.